Freight Rate Prediction — Figure Book

Every chart produced by the pipeline, with what it shows, what it found and the decision it supports. Figure IDs match the files in `outputs/figures/`. Use this to choose which charts go into the report and video.

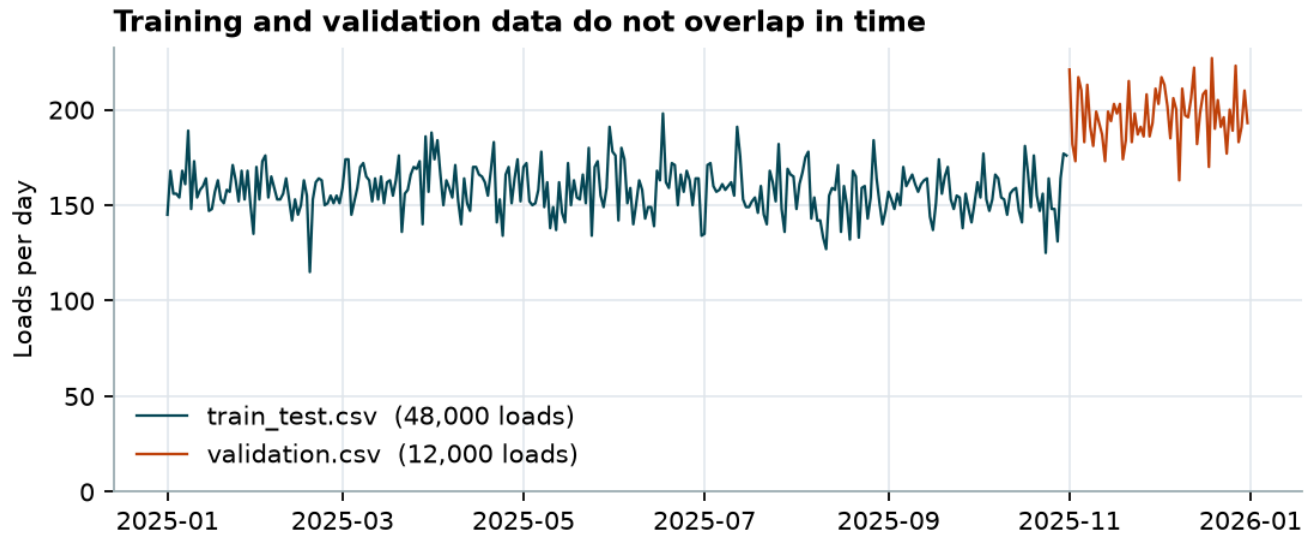
Contents

1. 1. Exploratory data analysis (20 charts)
2. 2. Model comparison (6 charts)
3. 3. Robustness checks (6 charts)
4. 4. Diagnostics of the chosen model (8 charts)
5. 5. Required deliverable: December chart from score.py

1. Exploratory data analysis

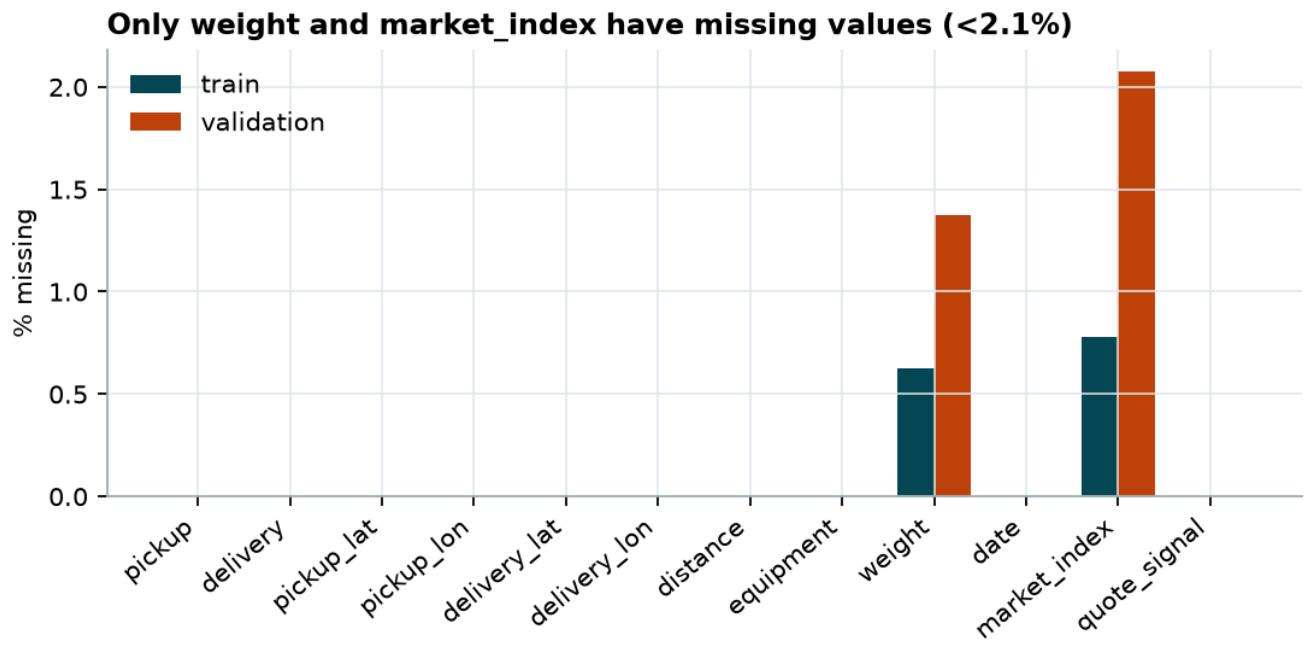
What the data looks like, what is wrong with it, and what that implies.

Data coverage over time eda/01_timeline.png



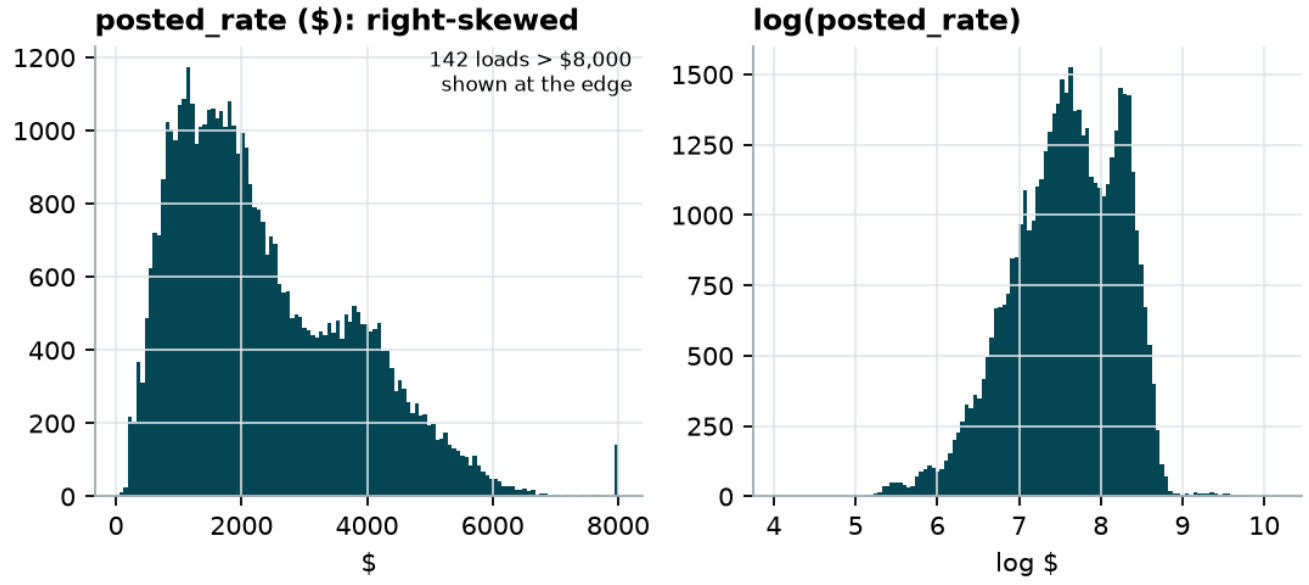
Shows	Number of loads per day in each file.
Finding	Training covers 01 Jan – 31 Oct 2025; validation covers 01 Nov – 31 Dec 2025. No overlap; volumes are steady (~160/day train, ~200/day validation).
Decision	Validation must be time-based (train on the past, score the future). A random split would leak same-day market conditions into training.

Missing values by column eda/02_missing.png



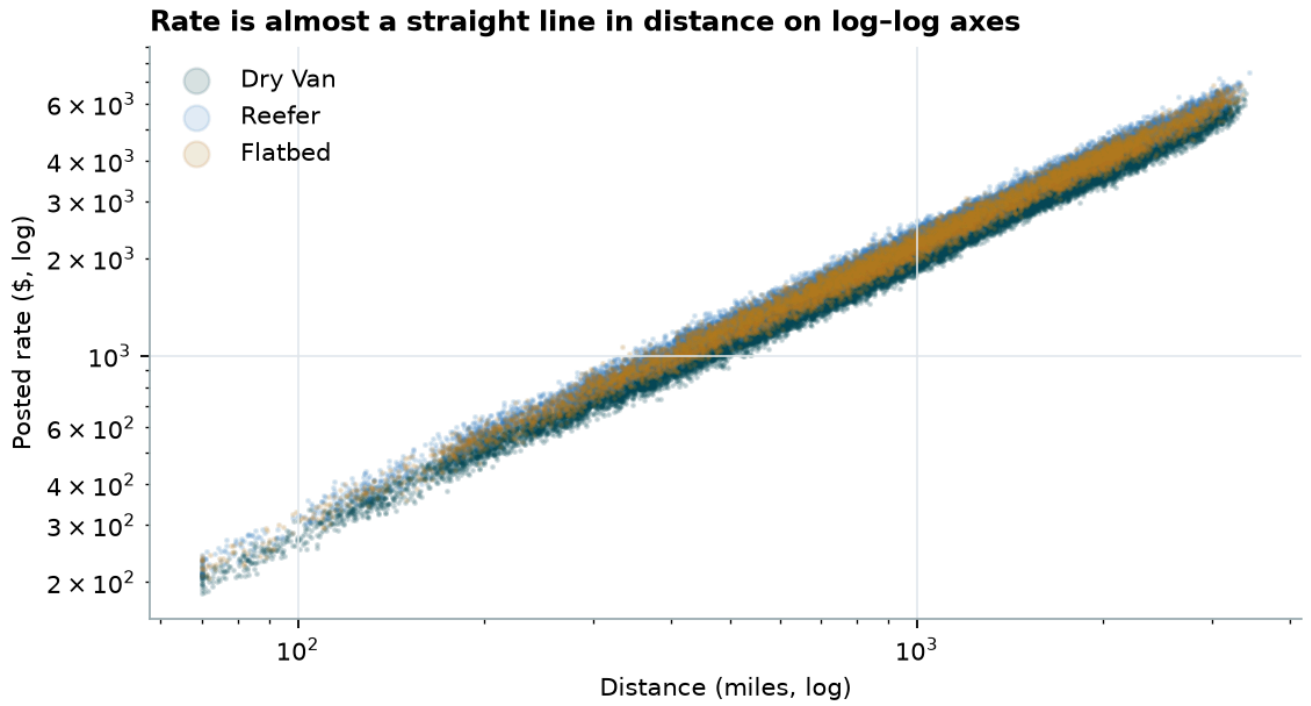
Shows	Share of missing values per column in each file.
Finding	weight: 300 train / 165 validation missing; market_index: 374 / 249. Every other column is complete, including the target.
Decision	No rows dropped for missing features. market_index -> that day's median (it is a market-wide signal); weight -> left as NaN for LightGBM (native handling) plus a missing flag, and equipment median for the linear stage.

Target distribution eda/03_target.png



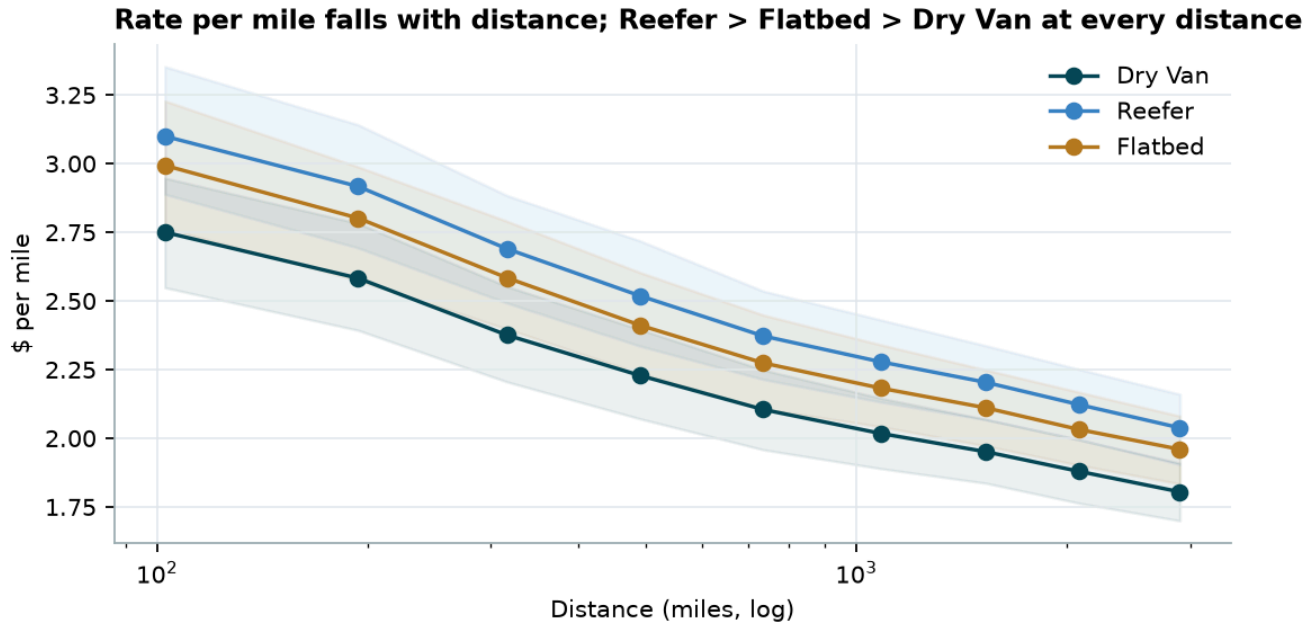
Shows	Histogram of posted_rate on the dollar scale and on the log scale.
Finding	Range $57 – $25,533, median $2,031. Strongly right-skewed with a long tail of extreme values.
Decision	Model log(rate): errors become relative (a $50 miss matters more on a $300 load than a $3,000 one), multiplicative effects (equipment premium, market index) become additive, and the tail has less leverage.

Rate vs distance (log–log) eda/04_rate_vs_distance.png



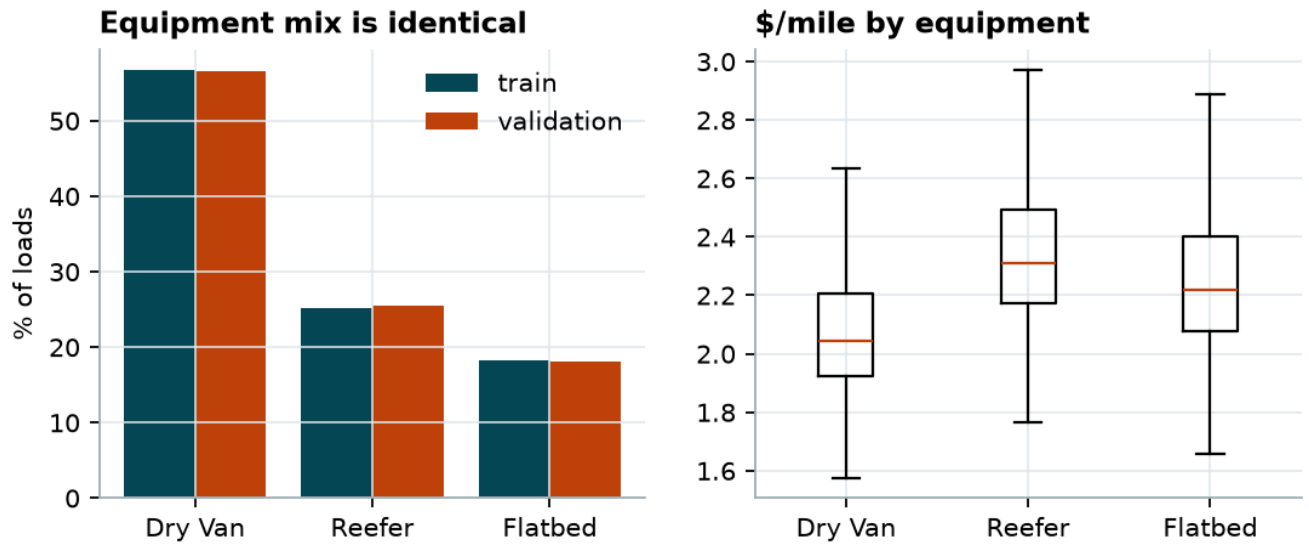
Shows	Each point is one load (flagged anomalous labels removed), coloured by equipment.
Finding	A straight line on log–log axes means a power law: rate ∝ distance^0.87. Distance alone explains 99% of the variance in log(rate).
Decision	log(distance) is the primary feature. A power below 1 means rate per mile falls with distance.

Rate per mile by distance and equipment eda/05_rate_per_mile.png



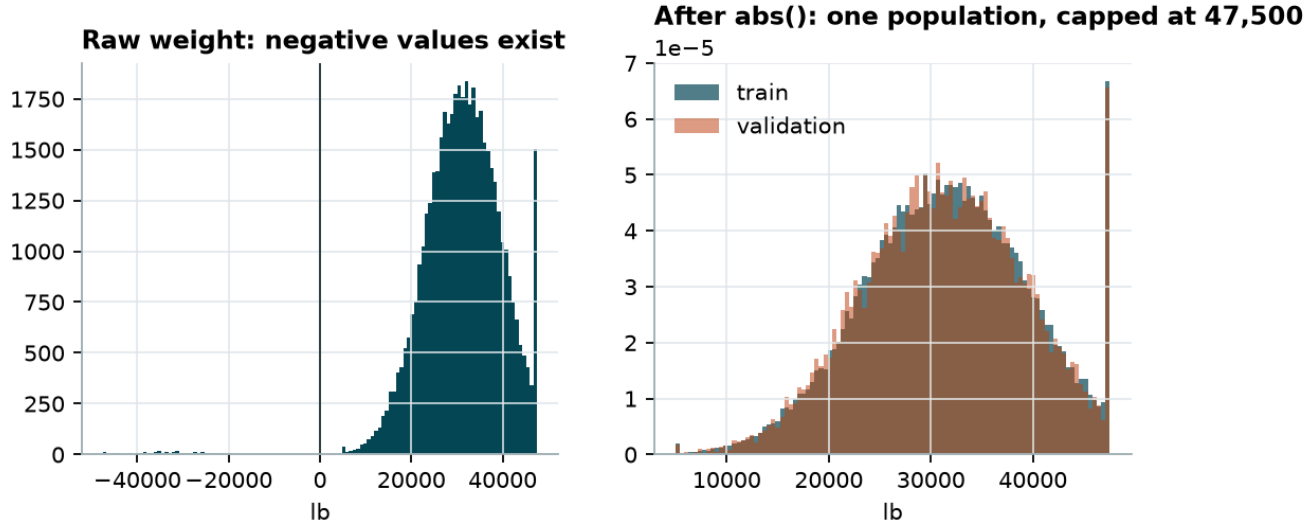
Shows	Median $/mile per distance band (line) with the 10th–90th percentile band (shaded).
Finding	Median $/mile: Dry Van 2.05, Flatbed 2.22, Reefer 2.31. The ordering holds at every distance, so equipment acts as a roughly constant % premium.
Decision	Equipment enters as a multiplicative premium (dummy variables on log scale).

Equipment mix and price level eda/06_equipment.png



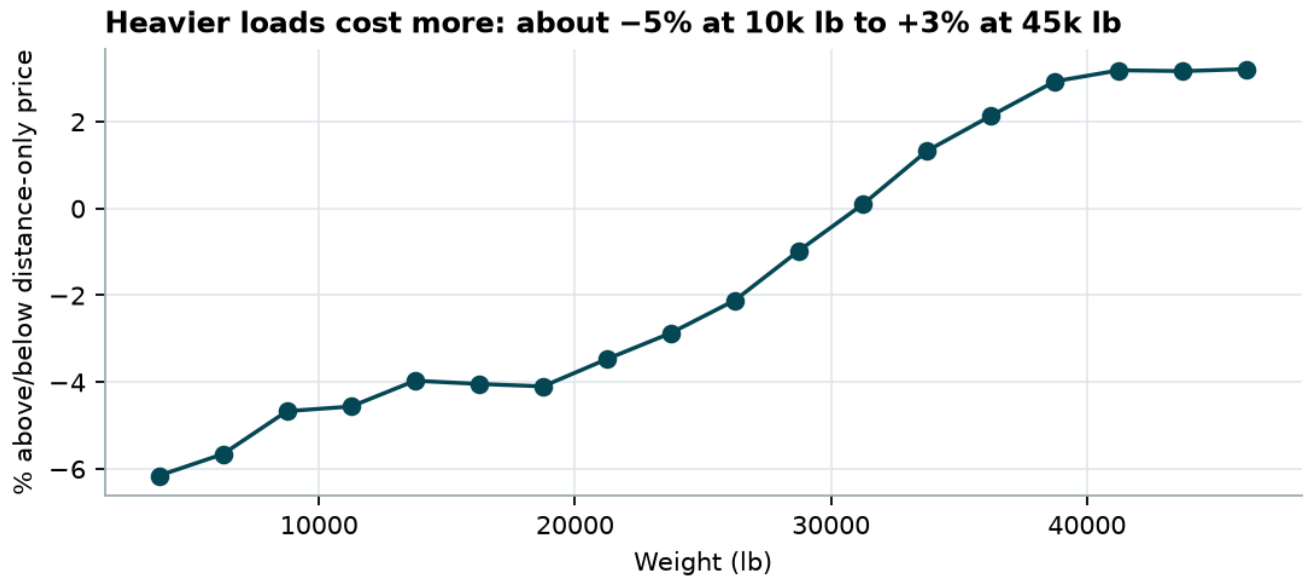
Shows	Left: share of each equipment type per file. Right: $/mile distribution (outlier labels removed).
Finding	Mix is ~57% Dry Van, 25% Reefer, 18% Flatbed in both files; no category appears only in validation.
Decision	No re-weighting needed for equipment.

Weight: sign errors and a cap eda/07_weight.png



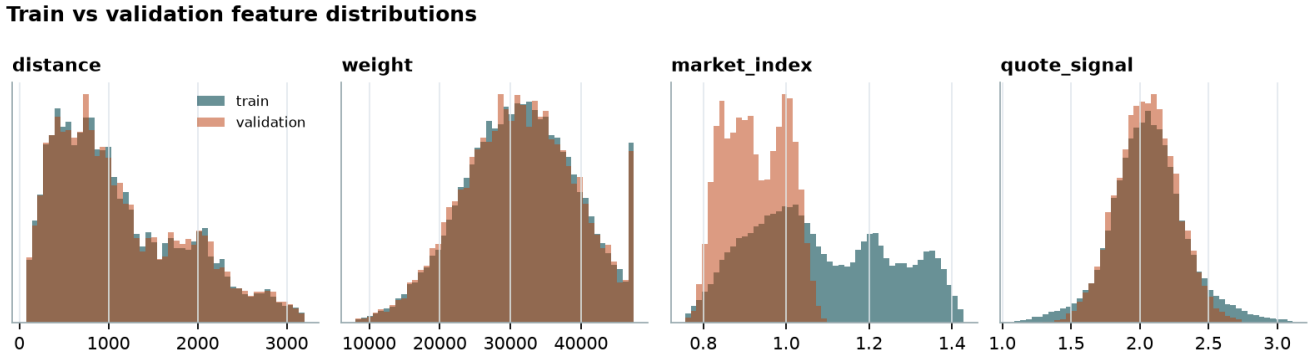
Shows	Left: raw weight including negatives. Right: weight after taking the absolute value.
Finding	292 train and 145 validation weights are negative, with magnitudes (median 31,822 lb) matching normal loads ⇒ sign errors, not a separate population. 2.5% of loads sit exactly at 47,500 lb (a cap).
Decision	Take abs(weight) and keep a 'was negative' flag. Keep the capped values (a legal-limit cap is plausible and the model sees it as 'heaviest').

Effect of weight on price eda/08_weight_effect.png



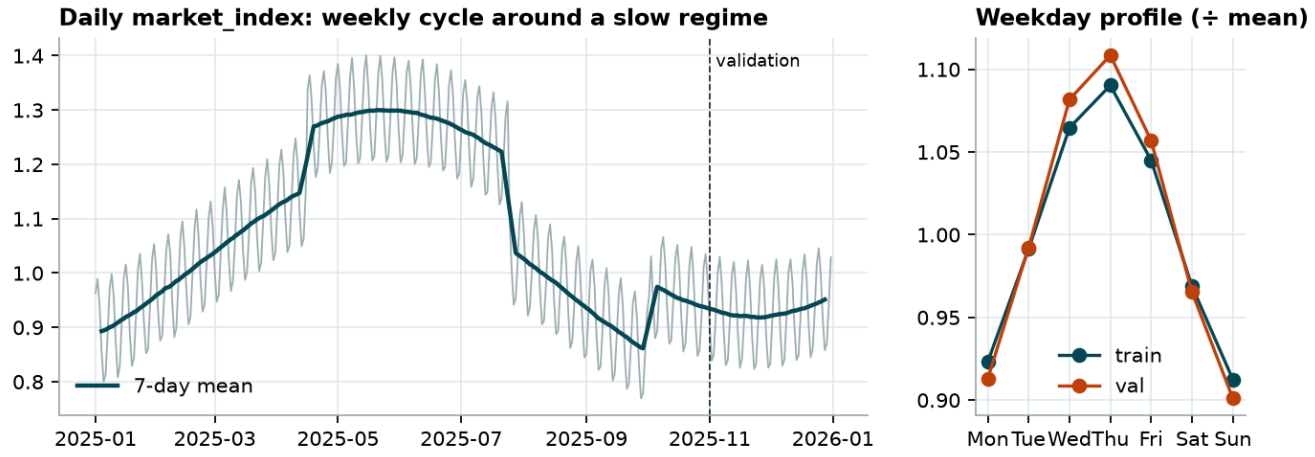
Shows	Mean residual of a distance-only fit, by weight band (shown as % difference).
Finding	Smooth, monotonic, roughly logarithmic relationship. Weight is a real but modest driver.
Decision	Use log(weight) in the linear stage and raw weight in the tree stage.

Train vs validation: covariate shift eda/09_shift.png



Shows	Density of each numeric feature in train_test.csv and validation.csv.
Finding	distance, weight and quote_signal match closely. market_index is the exception: validation sits in a narrow low band (0.72–1.10) vs 0.68–1.47 in training — but fully inside the training range.
Decision	Validation inputs are covered by training data, so no feature has to be extrapolated except time itself.

Market index over time eda/10_market_index.png

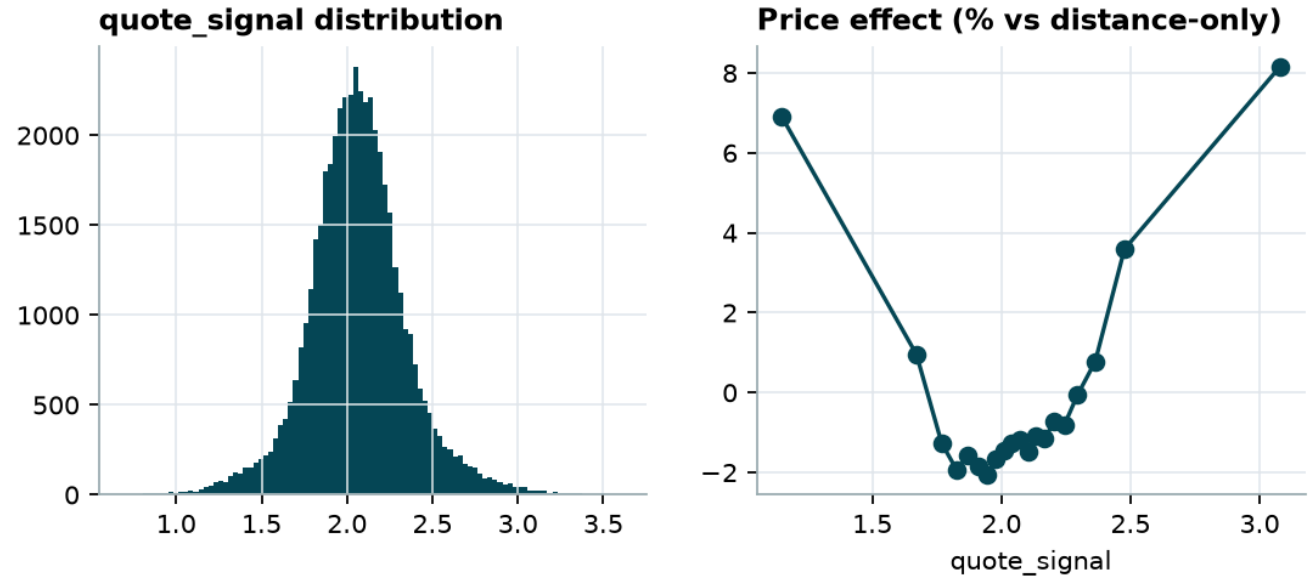


Shows Left: daily median market_index (grey) and 7-day mean. Right: average by weekday relative to the period mean.

Finding Two components: (1) a slow regime — rises Jan–May, plateaus, falls Aug–Sep, flat Oct–Dec; (2) a very regular 7-day cycle peaking Wed–Thu (±12%) that is identical in train and validation. Within a single day loads differ by only ~0.025.

Decision Treat market_index as a market-wide daily signal: impute by daily median and also give the model each load's deviation from it.

quote_signal eda/11_quote_signal.png

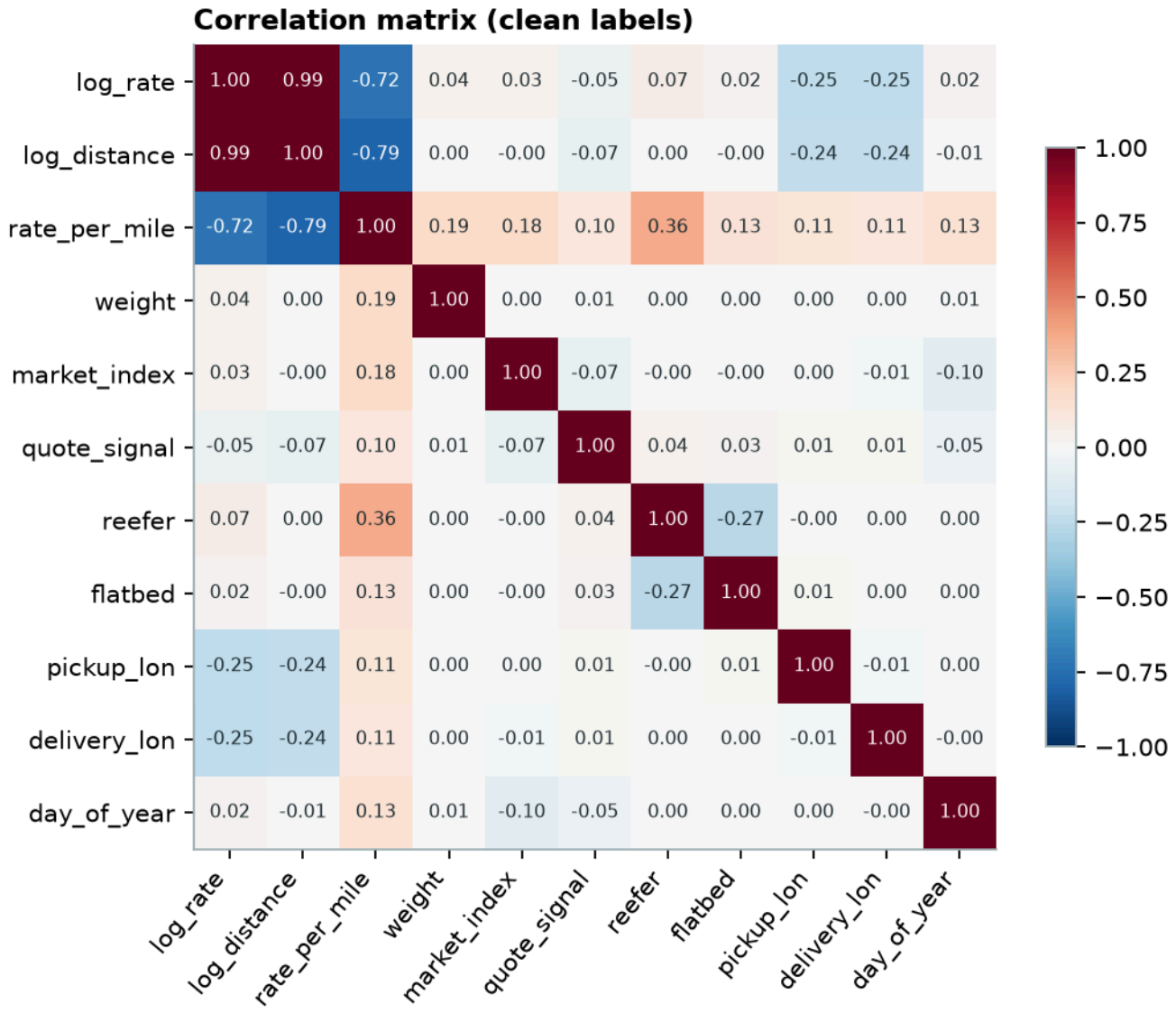


Shows Left: distribution. Right: mean residual of a distance-only fit by quote_signal vigintile.

Finding Correlation with $/mile is only 0.10. The effect is U-shaped: almost flat (about −2% to +1%) for the middle 80% of loads, rising to +4–8% only in the thin tails (quote_signal < 1.6 or > 2.5). A linear term cannot capture a U-shape.

Decision Keep it: its linear coefficient is near zero, but the tree stage can learn the U-shape.

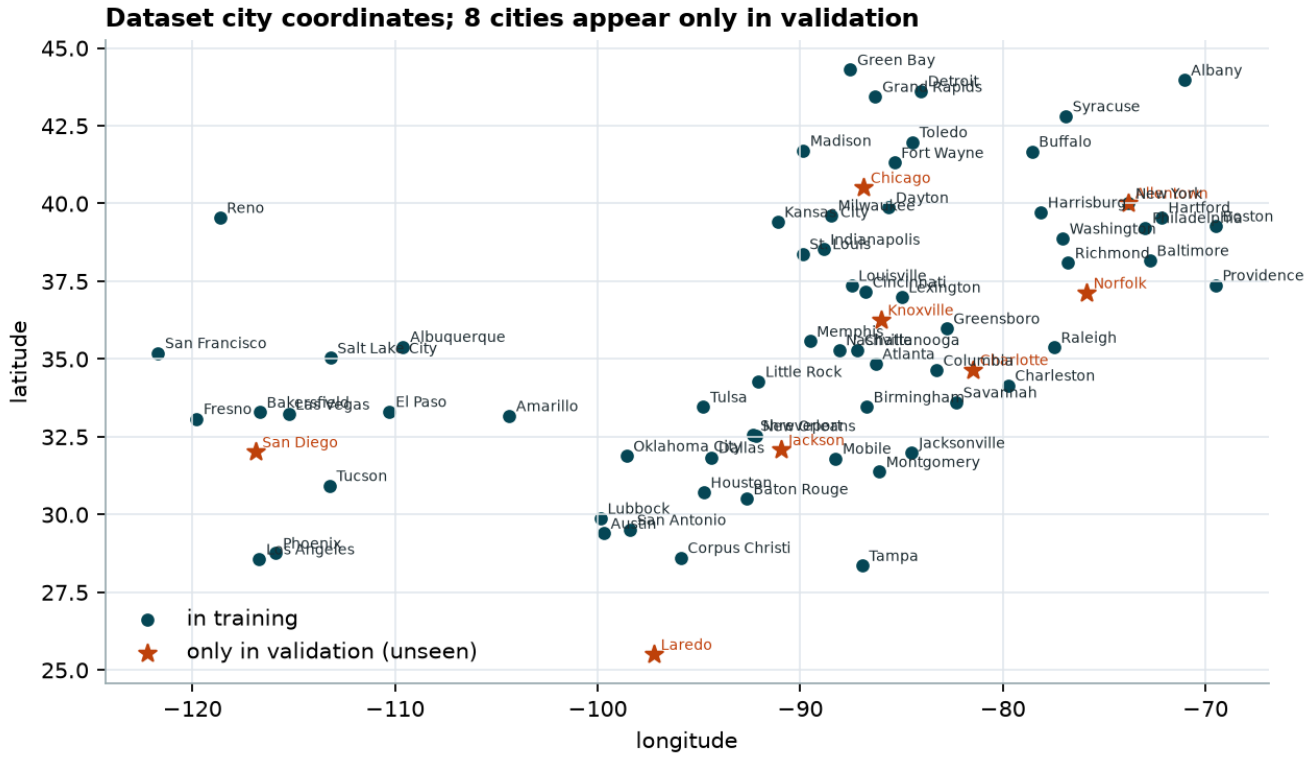
Correlations between features and target eda/12_correlations.png



Shows Pearson correlation between numeric features, log(rate) and rate per mile.

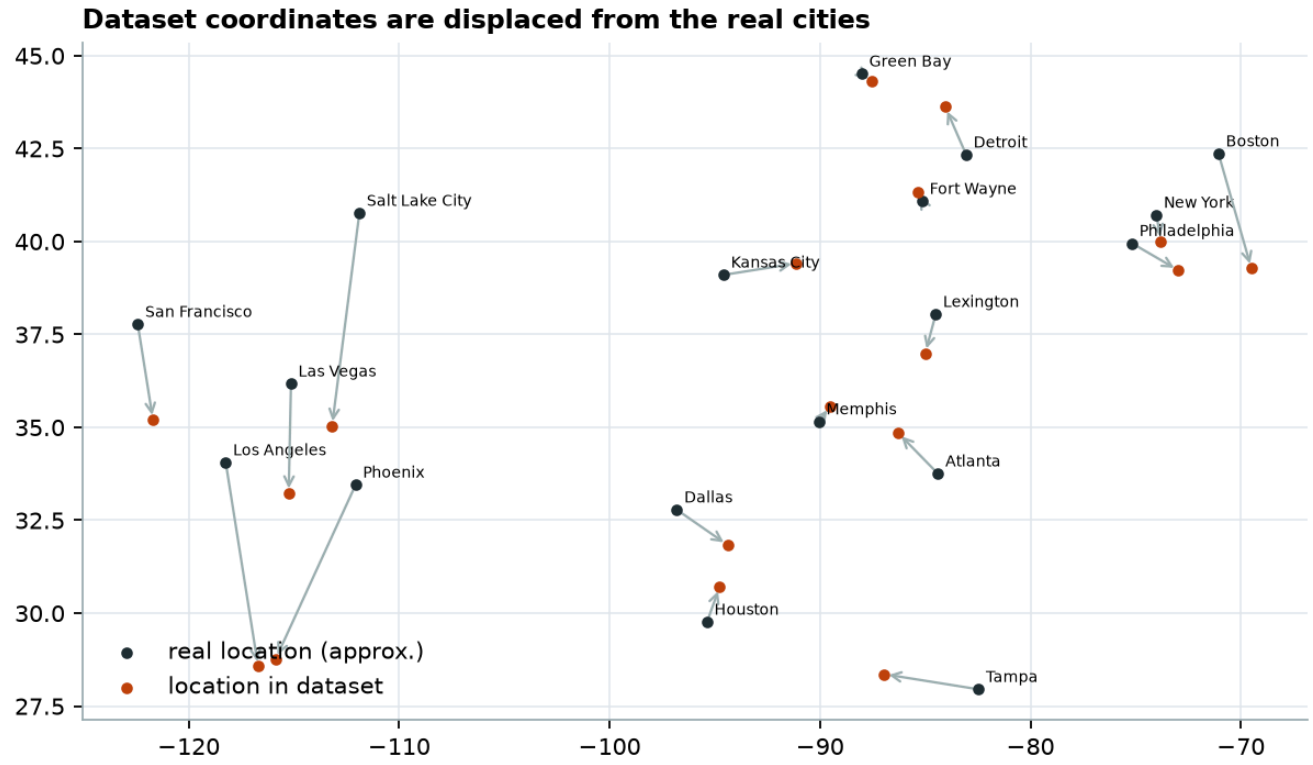
Finding log_distance dominates log_rate (r = 0.99). Among the rest, market_index, weight and equipment correlate with $/mile; features are not strongly correlated with each other, so multicollinearity is not a concern for the linear stage.

City locations and unseen cities eda/13_city_map.png



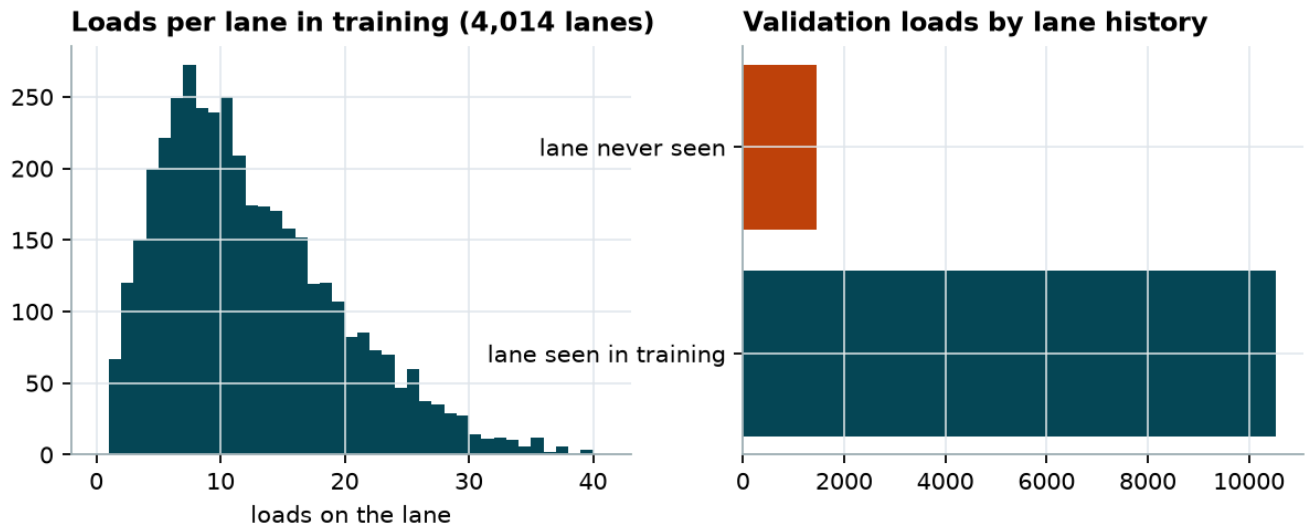
Shows	Each city's coordinates as given in the data. Stars = cities that never appear in training.
Finding	72 cities. Unseen in training: Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego. 1,447 validation loads (12%) touch at least one of them.
Decision	City name cannot be a model feature (no training rows for 8 of them). Describe lanes by coordinates, distance and circuity instead, which exist for every city.

Are the coordinates real? eda/14_coordinates_vs_real.png



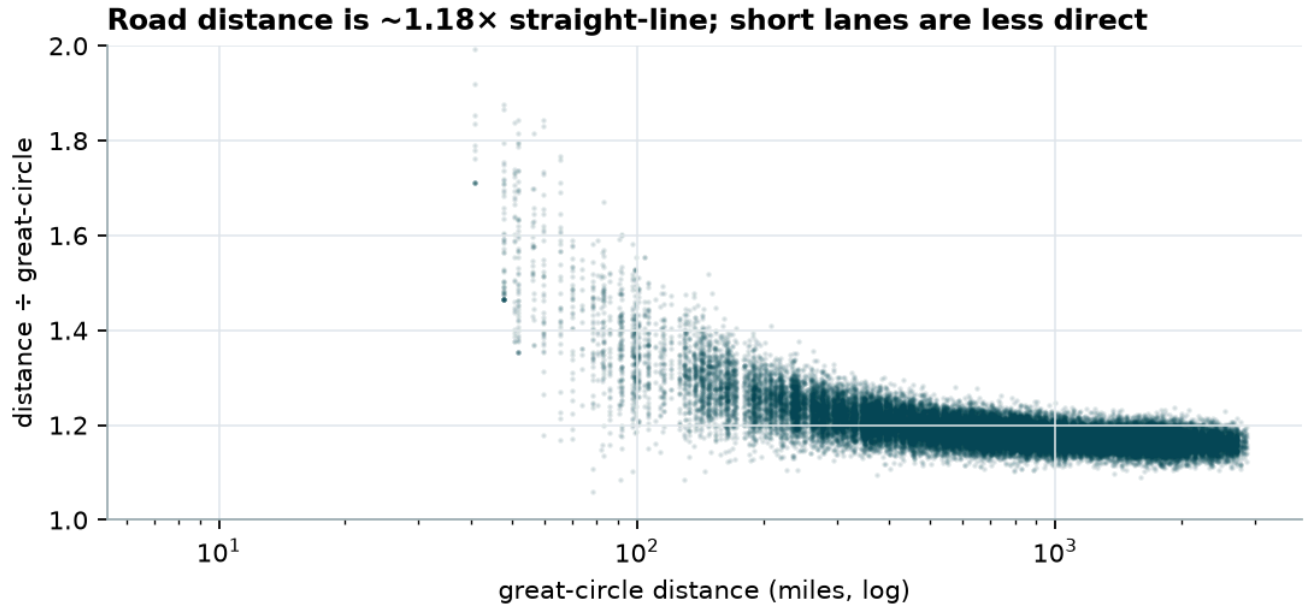
Shows	Arrows from each city's approximate real location to where the dataset puts it (18 well-known cities).
Finding	Displacements are large (median 144 miles; Phoenix is moved ~395 miles). So coordinates are synthetic/perturbed, not real geography. However each city has exactly one coordinate pair, and the distance column is consistent with them (median distance ÷ great-circle = 1.18, a typical road-circuity factor).
Decision	Treat coordinates as dataset-internal location signals, not real geography. Do not join external data (e.g. real regional rate indices) on them. This corrects the earlier report wording.

Lane coverage eda/15_lanes.png



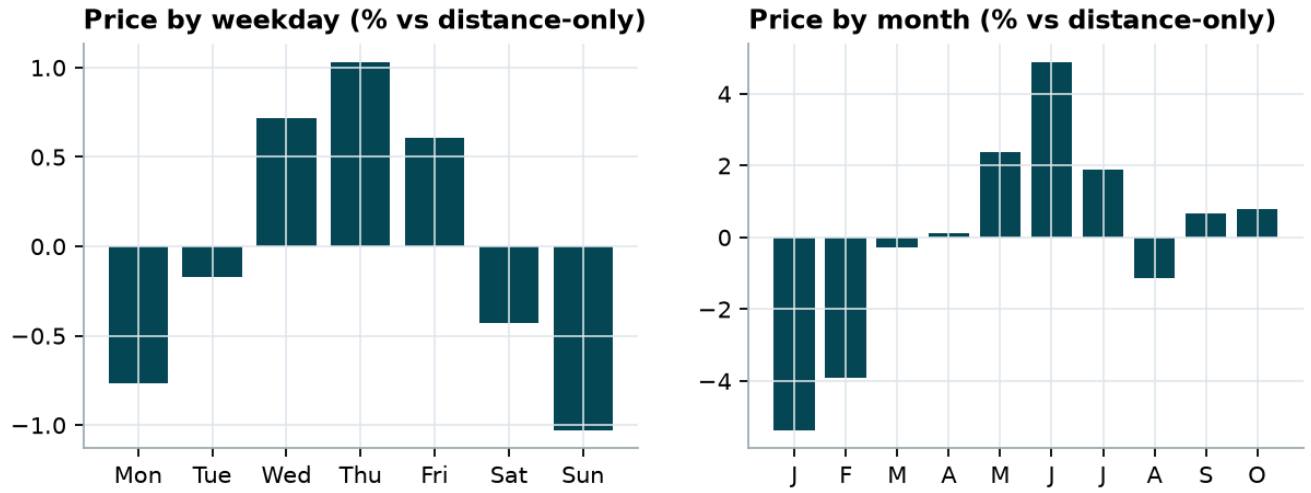
Shows	Left: how many training loads each pickup→delivery lane has. Right: validation loads on lanes with/without training history.
Finding	Median lane has only 10 loads, and 12% of validation loads are on lanes never seen in training.
Decision	Lane-level target encoding would be noisy (few loads per lane) and unavailable for 12% of validation. Use smooth geographic features that generalise between lanes.

Circuity (road vs straight-line distance) eda/16_circuity.png



Shows	Ratio of the distance column to the straight-line distance between the coordinates.
Finding	Median 1.18; rises above 1.4 only for short lanes (<150 miles straight-line). No impossible values (ratio < 1).
Decision	The distance column is trustworthy. Circuity is added as a feature for the tree stage.

Calendar patterns in price eda/17_weekday_month.png

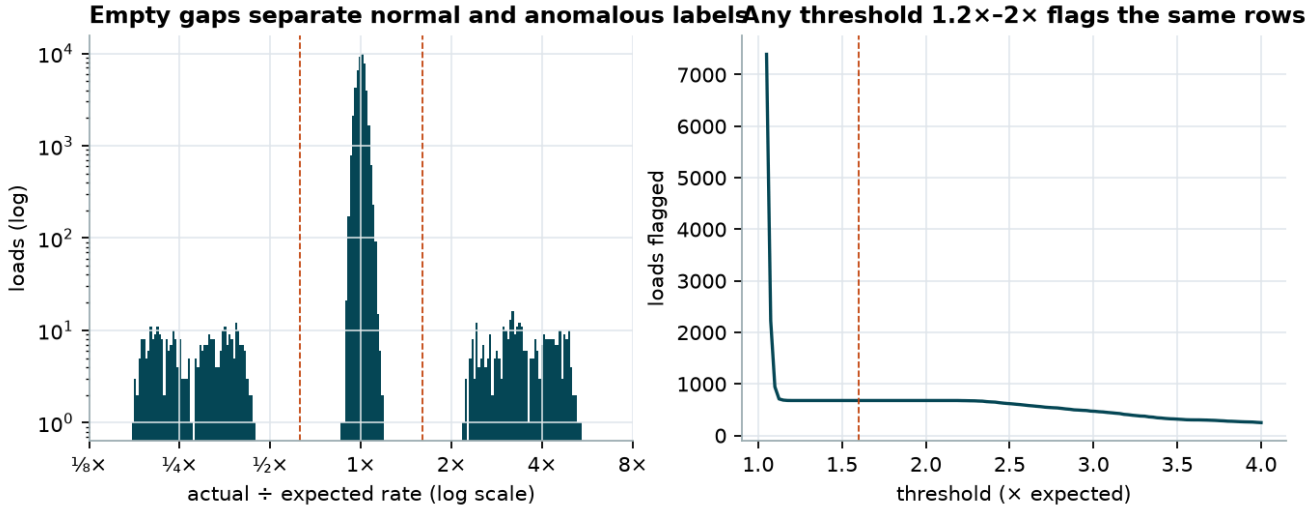


Shows Mean residual of a distance-only fit by weekday and by month.

Finding Weekday pattern mirrors the market_index weekly cycle (Wed–Thu highest). Month pattern rises through spring, peaks in June, dips in Aug–Sep and recovers in Oct.

Decision Both are largely explained by market_index; the remainder is studied in the date-effect analysis (validation section).

Anomalous price labels eda/18_outliers.png

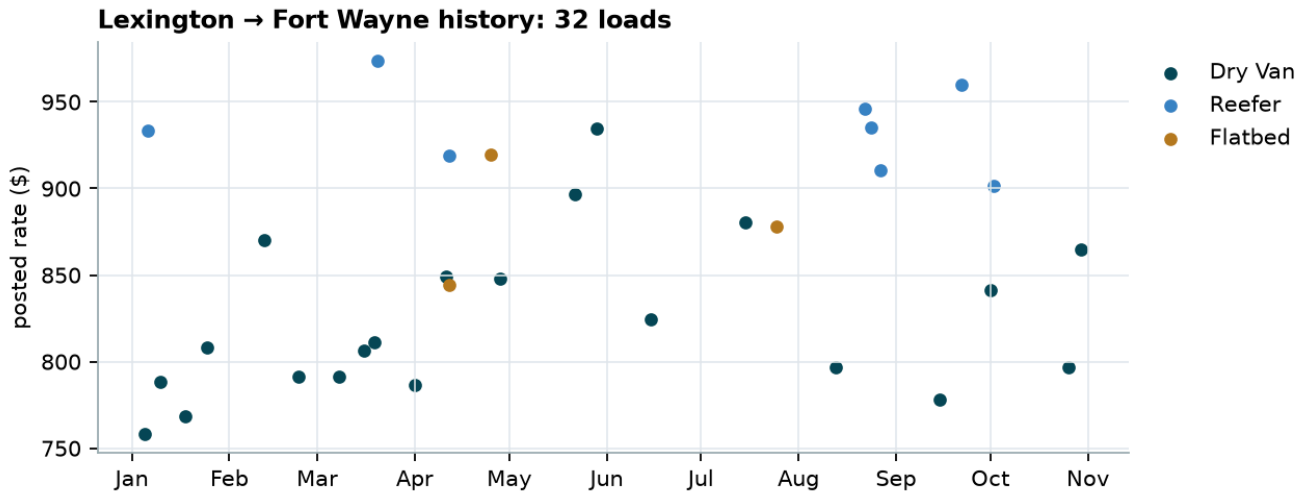


Shows Left: ratio of each actual rate to a robust (Huber) log-linear expectation. Right: number of loads flagged as the threshold changes.

Finding Normal loads all fall within ±19% of expectation. Then there is an empty gap, and 677 loads (1.4%) sit at 0.2–0.5× or 2–5×. Every threshold from 1.20× to 2.19× flags exactly these rows. They are spread evenly across months, equipment types and distances, and their features look normal.

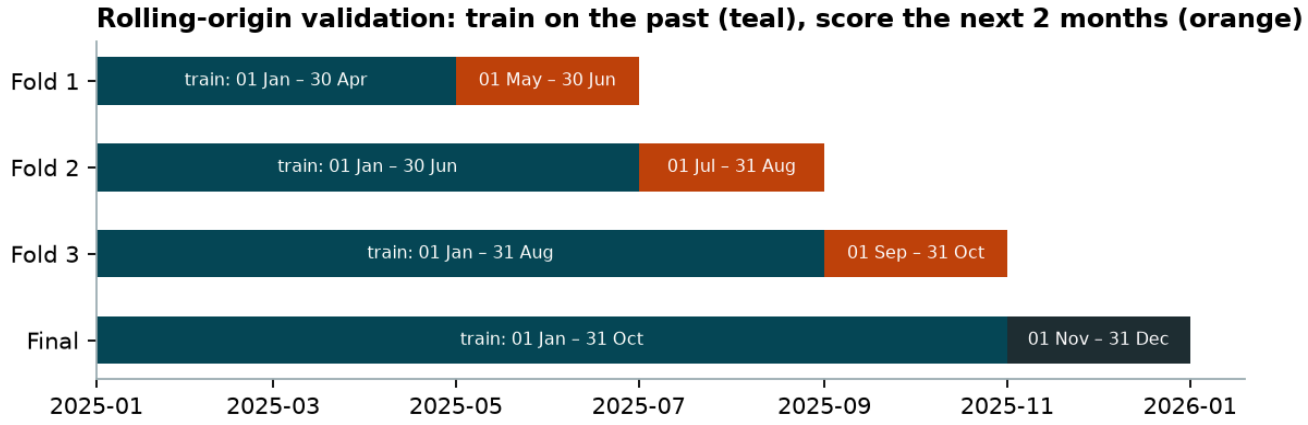
Decision Treat them as anomalous labels (likely data-entry or unit errors, but not proven) and exclude them from training; the threshold choice does not matter. Still report metrics with them included.

History of the December chart lane eda/19_december_lane.png



Shows	Every training load on Lexington → Fort Wayne (the fixed lane in the December chart).
Finding	32 loads, $758–$974, mixed equipment and weights (dry van only: 21 loads, $758–$934).
Decision	Sanity reference for the December chart: predictions around $820–$860 for a 32,000 lb dry van sit inside this historical range.

Validation design eda/20_cv_design.png

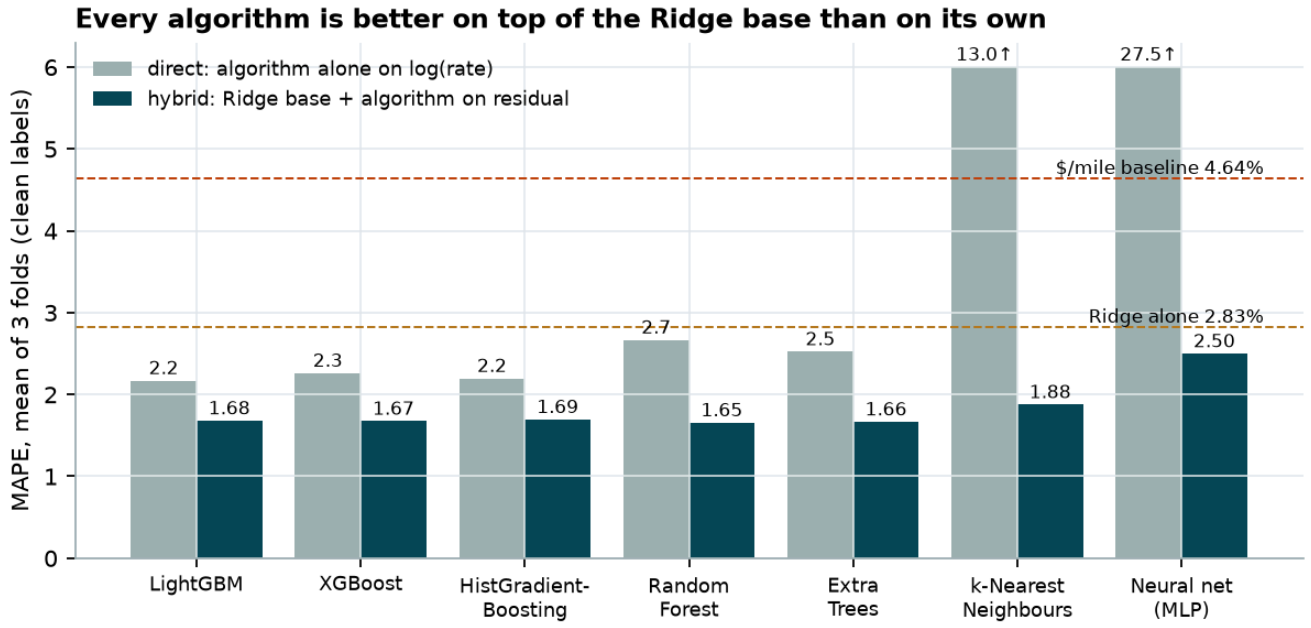


Shows	Three expanding-window folds plus the final fit. Dates are inclusive.
Finding	Each fold reproduces the real task: everything before a cut-off is training data, the next two calendar months are scored. The final model is trained on all of Jan 1 – Oct 31 and predicts Nov 1 – Dec 31, the same 2-month horizon.
Decision	Same folds, rows and metrics for every model compared; outlier detection is re-fitted inside each fold on training months only.

2. Model comparison

21 candidates on the same three time-based folds, same rows, same metrics.

Direct vs hybrid, for 7 algorithm families models/01_direct_vs_hybrid.png

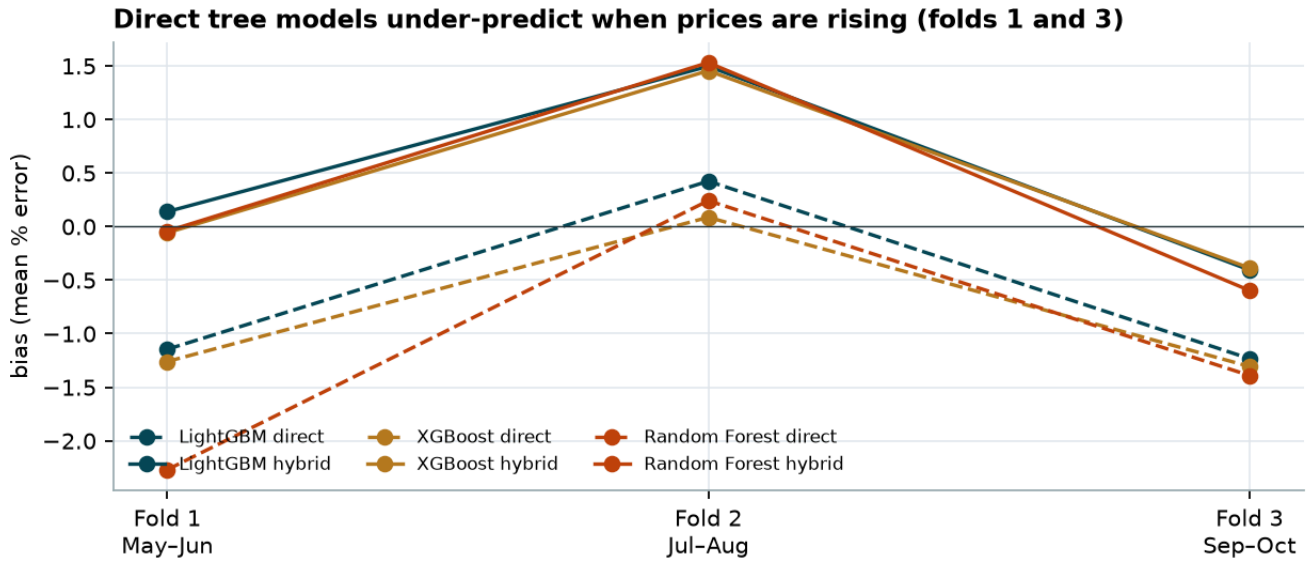


Shows Each algorithm trained (a) directly on log(rate) with all features including a time-trend feature, (b) on the residual of the Ridge base. Same folds, same rows, same metric. Bars clipped at 6% (k-NN and MLP direct are far worse).

Finding The hybrid wins for every family, by 0.5–1.0 points for tree models. Reason: trees cannot extrapolate. Given a trend feature, a tree predicts the same value for every date after its last split, so direct tree models under-predict later months (negative bias in folds 1 and 3, next chart). The Ridge base carries the trend forward; the tree only learns patterns that repeat (geography, interactions). k-NN and MLP direct fail badly: unscaled time extrapolation and no built-in log-linear structure.

Decision Use the hybrid structure for the final model.

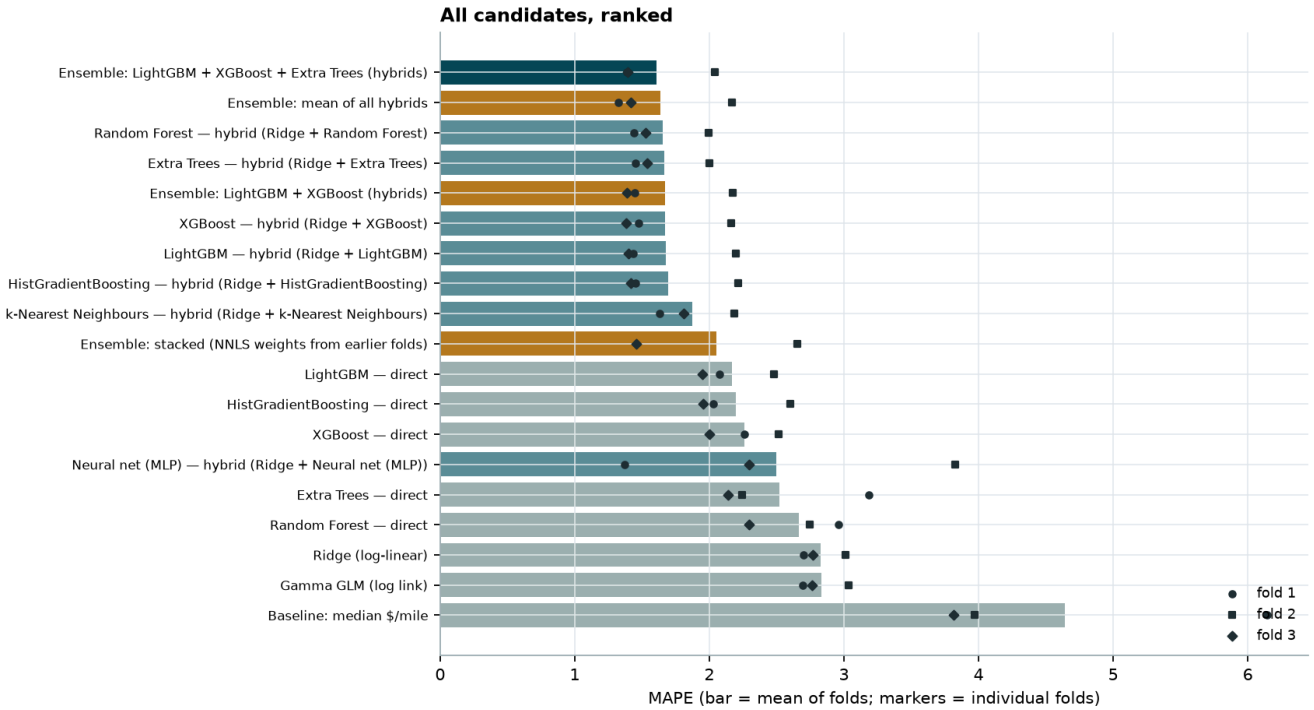
Why direct trees lose: bias models/02_bias_by_fold.png



Shows Mean signed % error per fold (+ = over-prediction) for three tree learners, direct (dashed) vs hybrid (solid).

Finding Direct tree models (all five) are 1.1–2.9% too low in folds 1 and 3: they freeze the price level at the end of training. Hybrids stay within ±0.7% there. In fold 2 every model over-predicts by ~1.5% because the trend estimated from Jan–Jun alone was too steep (see checks).

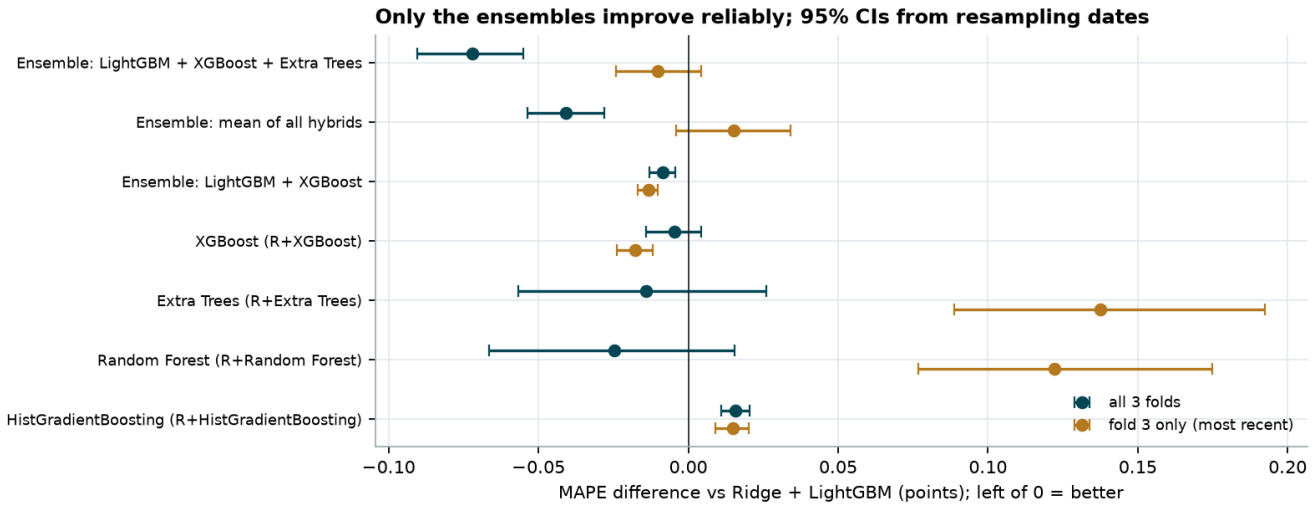
Leaderboard of all 21 candidates models/03_leaderboard.png



Shows Mean clean-label MAPE over the 3 time folds (bars) and each fold separately (markers). Teal = chosen ensemble, gold = other ensembles. The stacked ensemble has scores for folds 2–3 only (its weights need an earlier fold), so its bar is not directly comparable. Not shown (MAPE > 6%): Neural net (MLP) — direct, k-Nearest Neighbours — direct.

Finding Three tiers: (1) hybrids and their ensembles at 1.6–1.9%; (2) direct tree models at 2.2–2.7%; (3) linear/GLM at 2.8% and the $/mile baseline at 4.6%. Ridge and the Gamma GLM are indistinguishable, so the log-normal Ridge is not losing anything versus a GLM made for positive skewed targets.

Is the ensemble's gain real? models/04_ensemble_significance.png

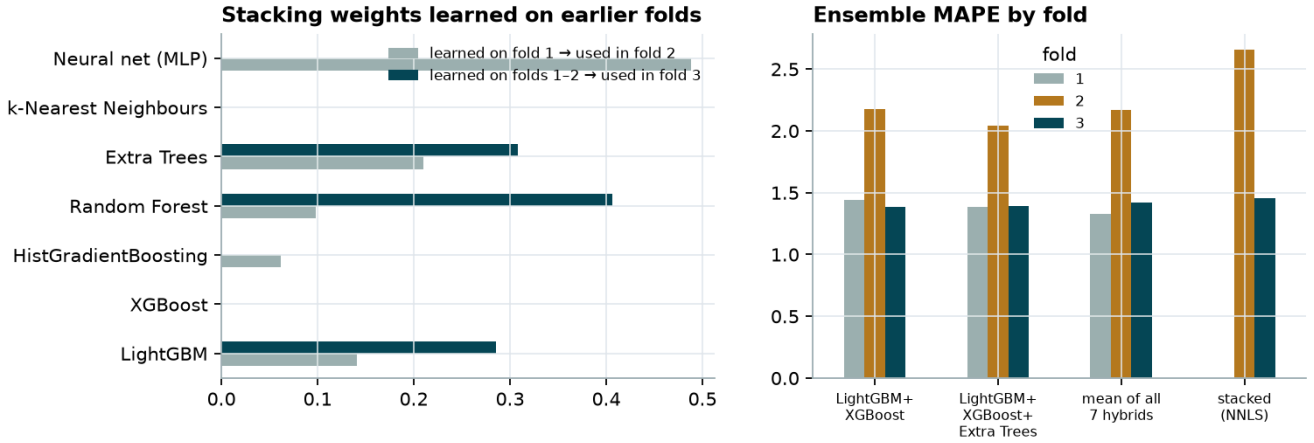


Shows Difference in MAPE between each candidate and the original Ridge + LightGBM, with 95% confidence intervals from a bootstrap over dates (2,000 resamples; whole days are resampled because loads on the same day share market conditions).

Finding LightGBM + XGBoost + Extra Trees: -0.072 points over all folds (CI -0.090 to -0.055, better on 73% of dates) and -0.010 on fold 3 (CI -0.024 to +0.004: no loss). Single learners are within ±0.02 points of LightGBM (XGBoost, HistGradientBoosting) or clearly worse on the most recent fold (Random Forest / Extra Trees alone lose 0.13–0.15 points). Combining tree types that make different errors (boosting vs randomised bagging) is what helps.

Decision Chosen model: Ridge base + equal-weight average of LightGBM, XGBoost and Extra Trees residual models. Rule applied: best mean across folds, provided it is not significantly worse on the most recent fold. Gain is modest ($2.0 MAE) at the cost of ~3× training time.

Simple averaging vs learned stacking models/05_stacking.png

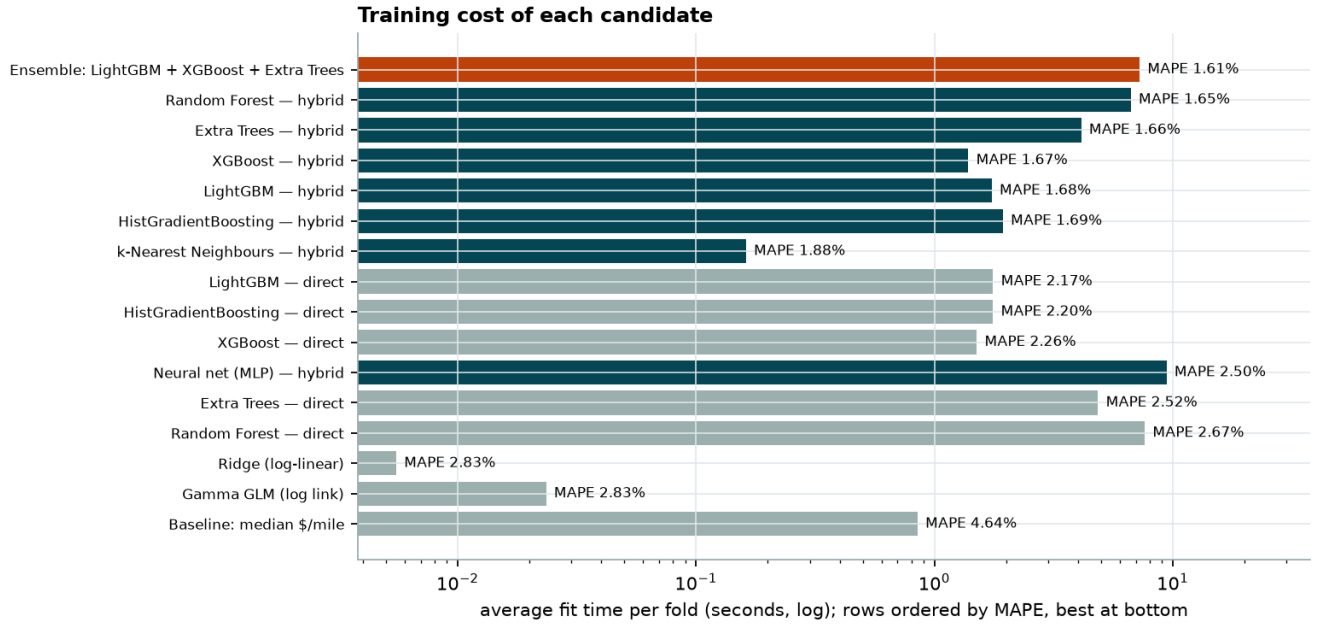


Shows Left: non-negative stacking weights learned walk-forward (only earlier folds' predictions). Right: fold MAPE of each ensemble (stacking has no fold-1 score because no earlier fold exists).

Finding Learned weights jump around between folds (e.g. MLP gets 38% from fold 1, 0% from folds 1–2) and the stacked ensemble is the worst ensemble. With only 2–3 folds the weights over-fit the specific market of the earlier months.

Decision Use fixed equal weights: nothing to over-fit.

Accuracy vs training cost models/06_accuracy_vs_cost.png



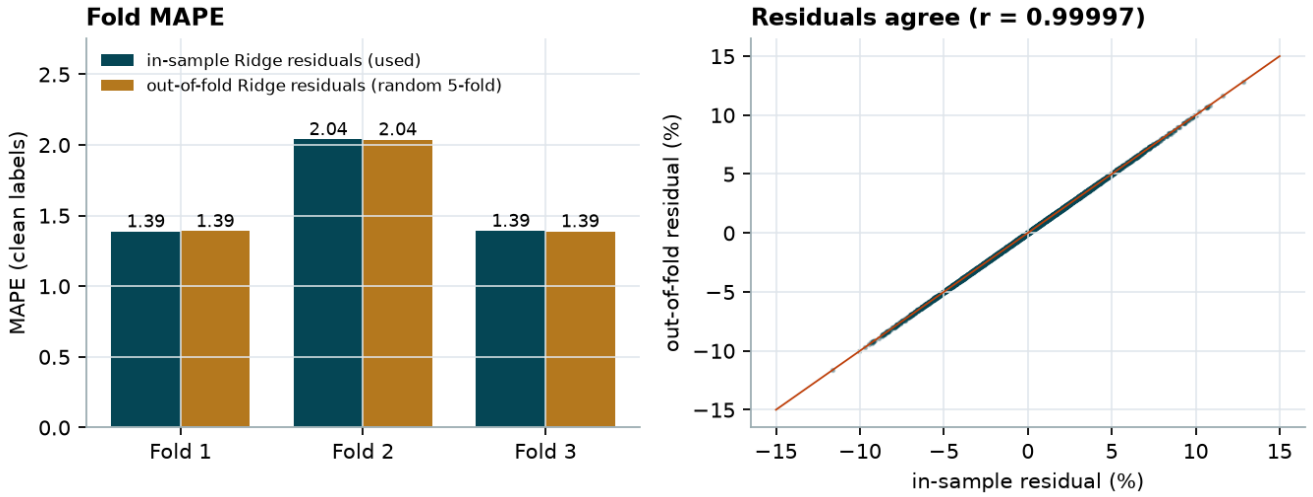
Shows Mean MAPE against fit time on a 2-core machine. (h) = hybrid, (d) = direct.

Finding The chosen ensemble trains in ~7 s per fold; still trivial for daily retraining.

3. Robustness checks

Each check answers one review question about leakage, assumptions or claims.

Is stage 2 trained on leaky residuals? checks/01_residual_leakage.png

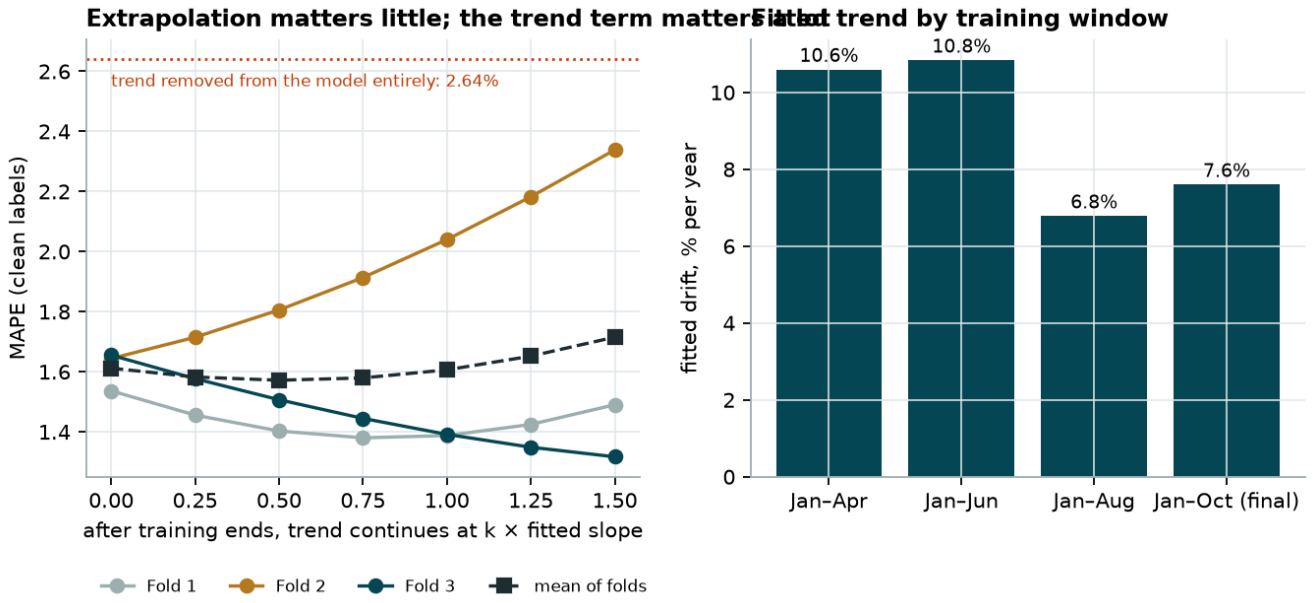


Shows Left: fold MAPE when the stage-2 models learn in-sample Ridge residuals (submitted) vs out-of-fold Ridge residuals (each row's residual from a Ridge fitted on the other 80% of rows). Right: the two kinds of residual for the same rows (5,000-row sample, final training set).

Finding Largest fold difference 0.003 points. The residuals are practically identical (correlation 0.99997, largest difference 0.10% of price): Ridge has 8 coefficients fitted on 15,000–47,000 rows, so it cannot memorise its own rows. (A time-blocked out-of-fold version, tried first, was worse — 1.9% on fold 1 — because holding out a whole period makes the residuals contain period-level errors, which is a different problem from leakage.) Both variants are always scored on later months.

Decision Keep the simpler in-sample residuals; document the check.

The time trend: what it does and how far to push it checks/02_trend.png

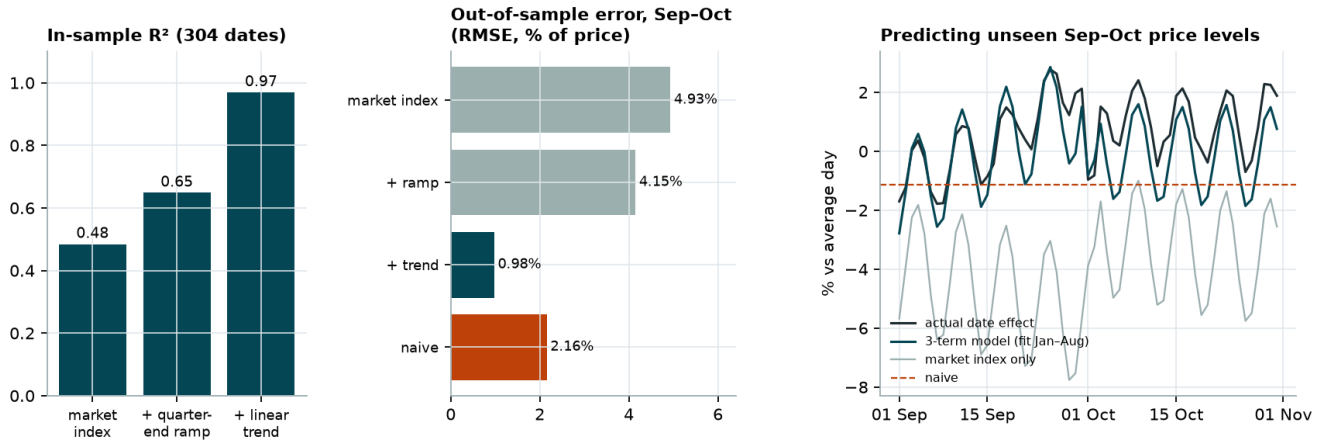


Shows Left: CV MAPE when, after the last training date, the trend continues at k × its fitted slope (k = 0 freezes the level at the training end; k = 1 is the submitted model). Dotted line: the same model trained without any trend term. Right: fitted drift for different training windows.

Finding Two separate effects. (1) Having a trend term at all is what matters: without it MAPE is 2.64% vs 1.61%, because the model otherwise prices every month at the training-period average level instead of the latest level. (2) How far to extend it beyond the training end matters little and depends on the period: across k = 0–1.5 the mean moves only 1.57–1.71%. Extending helps in fold 3 (1.65% → 1.39%) and hurts in fold 2 (1.64% → 2.04%), where the slope was estimated from Jan–Jun only (10.8%/yr, later revised to 7.6%/yr).

Decision Keep k = 1. The mean-optimal k = 0.5 is chosen from only three folds and differs from k = 1 by a few hundredths of a point, so tuning it would be over-fitting. The December chart is shown under k = 0, 0.5 and 1 (check 6) so the reader can see the assumption.

The date effect: how the 0.97 is defined and whether it carries forward checks/03_date_effect.png

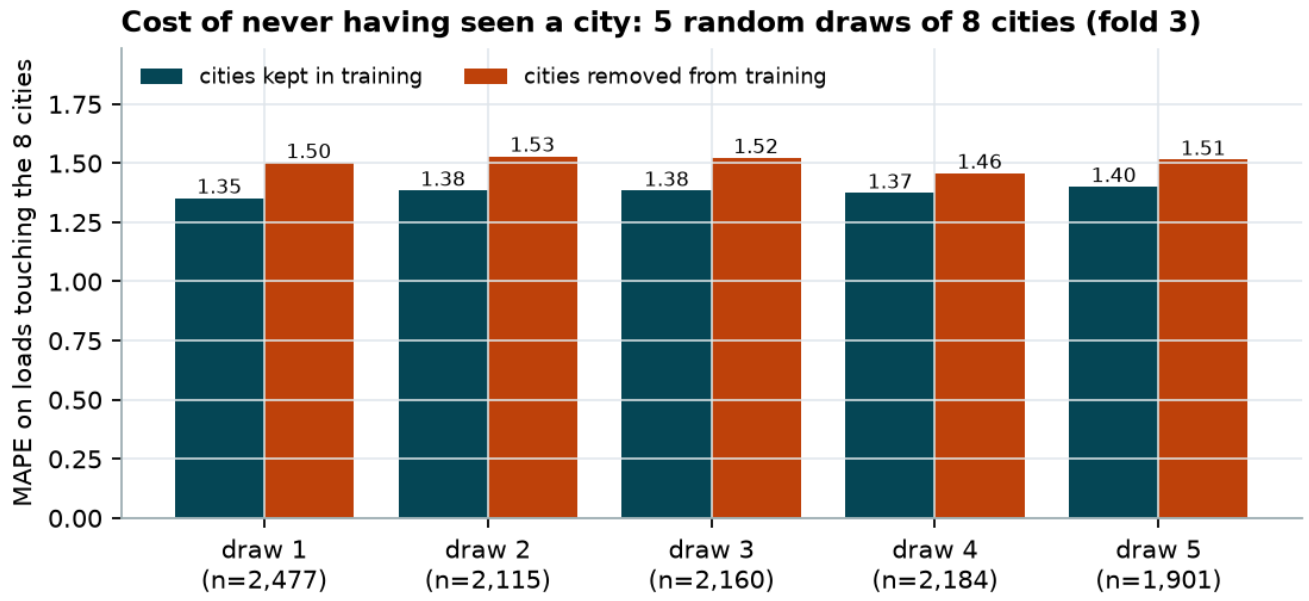


Shows Step 1: regress log(rate) on load features (log distance, equipment, log weight, quote_signal, the load's market-index deviation) plus one dummy per date (304 dates, clean labels). The dummy coefficients are the per-date price levels. Step 2: regress those levels on daily log(market_index), the quarter-end ramp and a linear trend. Left: ordinary in-sample R^2 of step 2. Middle/right: step 2 fitted on Jan–Aug dates only, then used to predict the Sep–Oct levels.

Finding In-sample R^2 rises 0.48 → 0.65 → 0.97. Out of sample, the 3-term model predicts the daily price level of unseen Sep–Oct within 0.98% RMSE, vs 2.16% for carrying the last level forward and 4.93% for market index alone (which gets the weekly wiggle but not the level). R^2 is not used out of sample because a 2-month window has little variance, which makes R^2 unstable.

Decision Describe 0.97 as an in-sample, descriptive figure; use the out-of-sample error as the evidence that the structure carries forward two months. It does not prove it holds in Nov–Dec.

Generalisation to unseen cities checks/04_unseen_cities.png

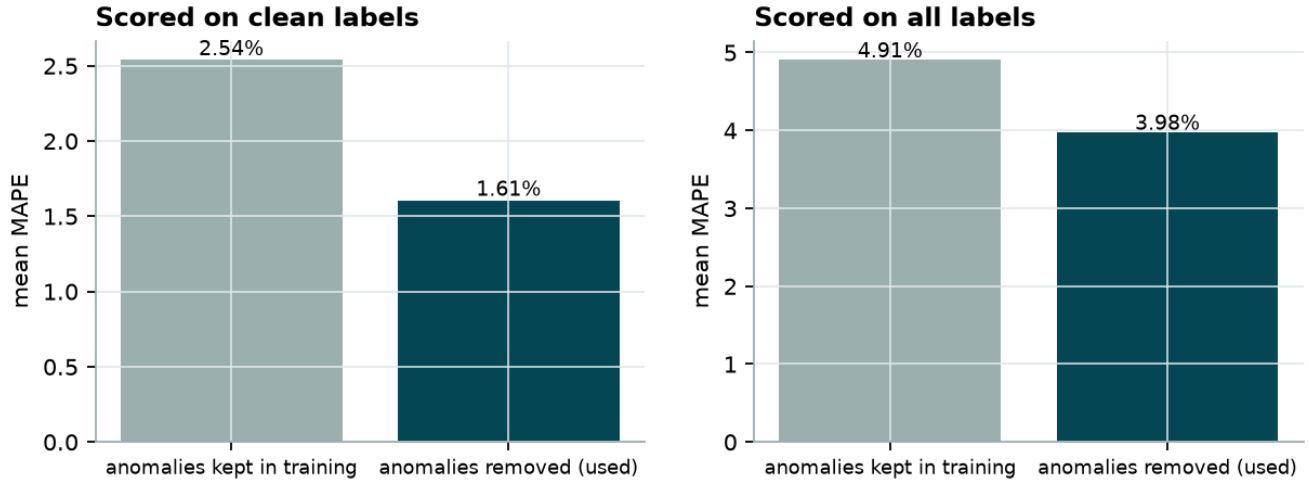


Shows For each draw, 8 random cities are chosen. The same fold-3 loads touching them are predicted by a model trained with those cities and one trained with every load touching them removed.

Finding Removing the cities raises MAPE by 0.13 points on average (range 0.08 to 0.15); MAPE stays at 1.46–1.53%. The random cities are not the 8 real unseen validation cities (those have no labels), so this tests unseen cities in general.

Decision Evidence (not proof) that predictions for the 1,447 validation loads touching new cities should be close in quality to the rest.

Does removing anomalous labels help? checks/05_outlier_impact.png

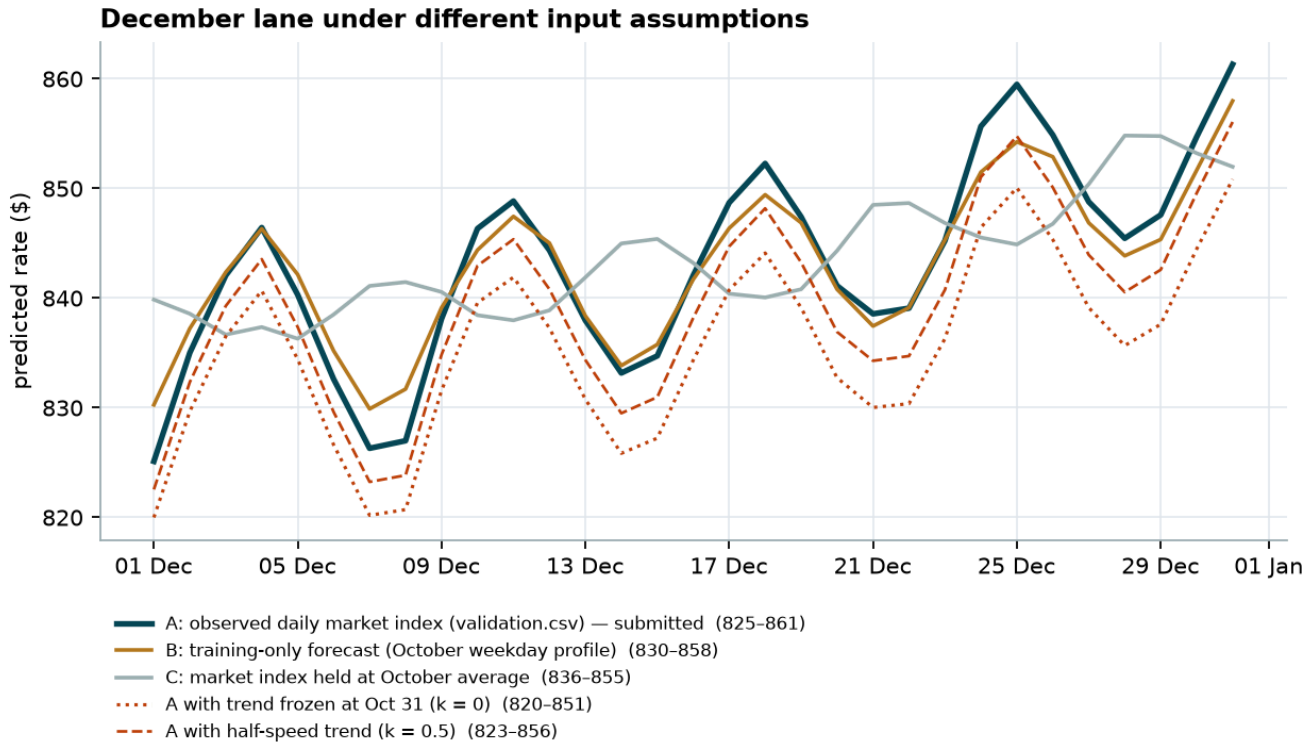


Shows Two-stage model trained with vs without the flagged labels (detector fitted per fold on training months), scored on clean labels and on all labels.

Finding Removing them improves accuracy on normal loads AND on all loads (anomalies included): a few labels at 3–5× or 0.2–0.4× pull a log-scale model's fit away from the bulk of the data, so keeping them makes every prediction slightly worse. The all-label error stays high either way because no model can anticipate a 4× label.

Decision Exclude them from training; report both metrics.

December chart: sensitivity to assumptions checks/06_december_scenarios.png



Shows The fixed Lexington → Fort Wayne dry-van load priced for every December day under alternative inputs. A (submitted) uses the market_index and quote_signal values supplied for those dates in validation.csv — input features only, never labels. B and C use training data only: October training rows for market_index and quote_signal, and no validation value enters any feature (inputs saved in december_scenario_inputs.csv). The last two curves vary the trend.

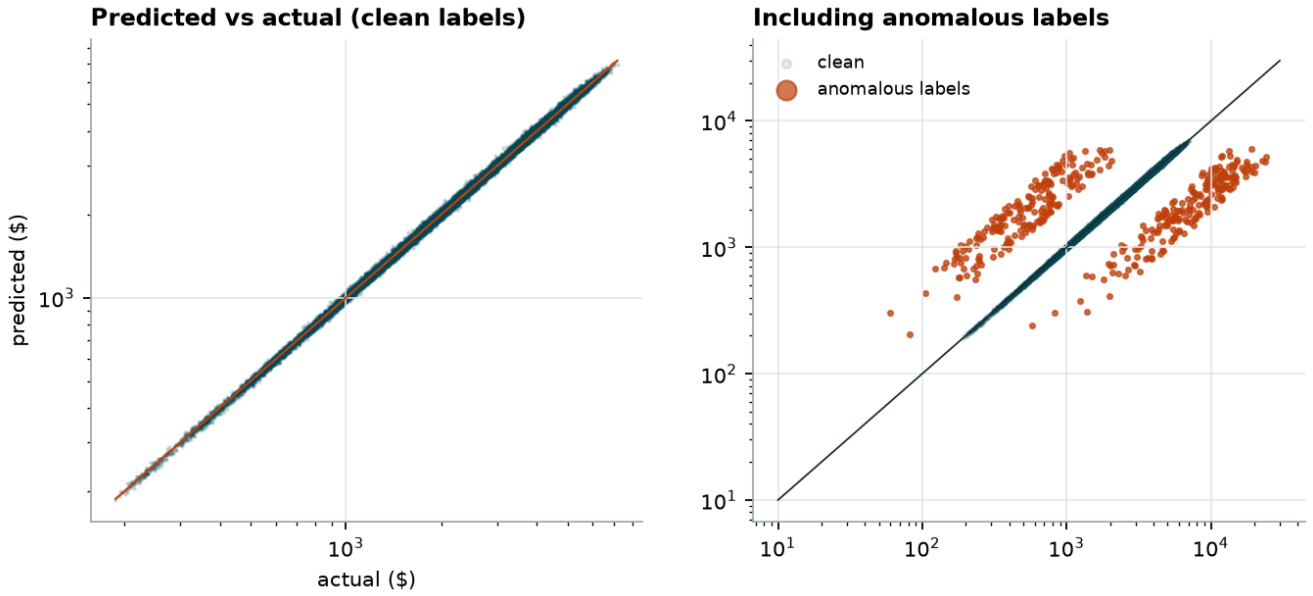
Finding All scenarios lie within a $41 band (~5% of the price). The training-only forecast B differs from A by -0.0% on average (largest daily gap $5), because the weekly market pattern is very regular. The trend assumption shifts the level by about 1%.

Decision Submit A: the assessment supplies these feature values for the December dates. If only the seven chart columns may be used, B is the alternative and gives nearly the same curve.

4. Diagnostics of the chosen model

Where the submitted model errs and whether its predictions look sane.

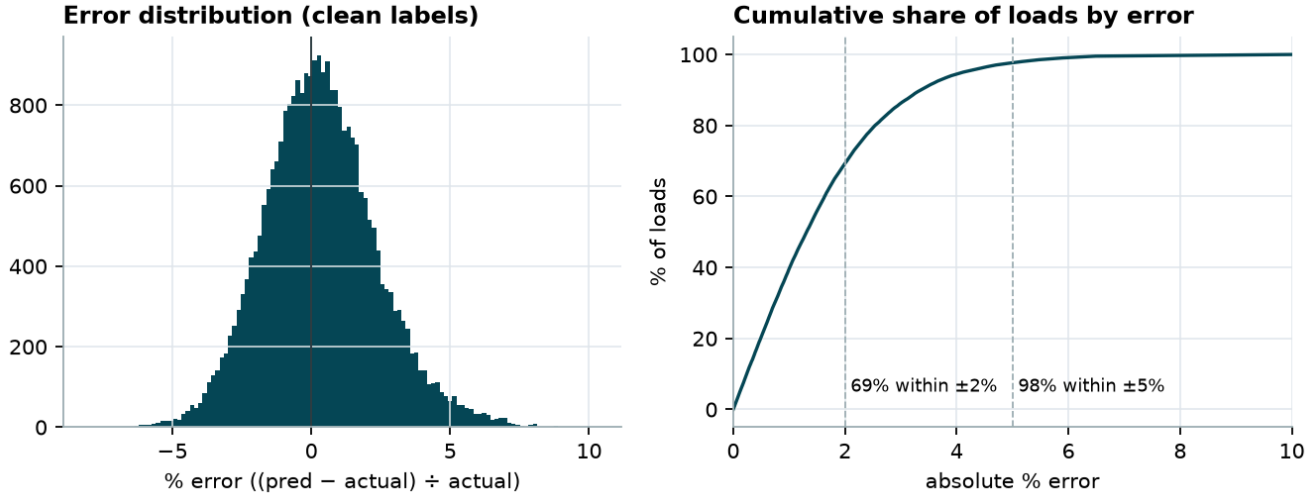
Predicted vs actual (out-of-fold) diagnostics/01_pred_vs_actual.png



Shows Every out-of-fold prediction from the 3 time folds (each predicted by a model that never saw its month).

Finding Clean labels hug the diagonal: 69% of loads within ±2%, 98% within ±5%, 100.0% within ±10%. The anomalous labels (right, orange) sit far off the line on both sides; the model predicts a normal price for them, which is the intended behaviour.

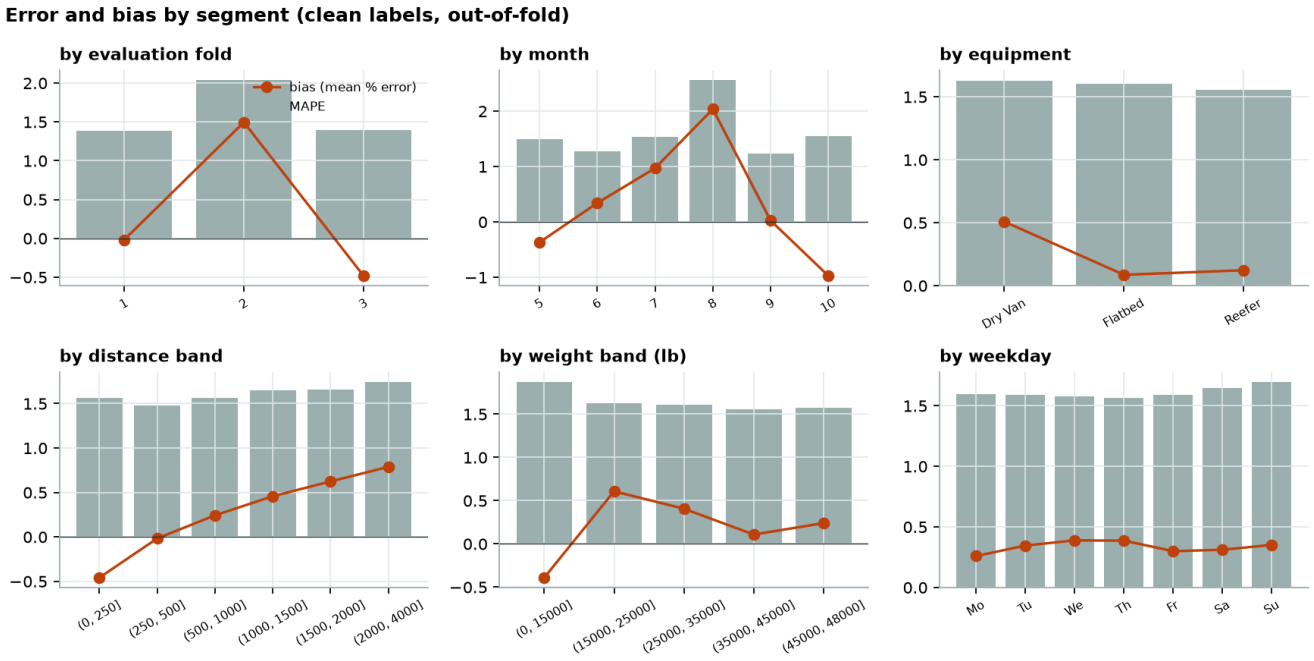
Size of errors diagnostics/02_error_distribution.png



Shows Left: histogram of signed % error. Right: cumulative share of loads with absolute error below x.

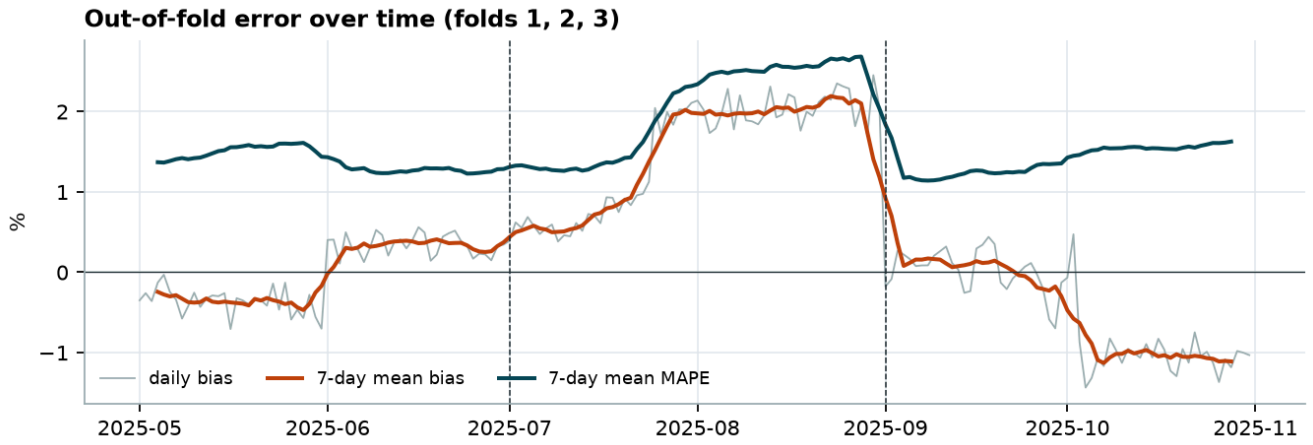
Finding Errors are symmetric and centred near zero (mean +0.33%, median +0.25%). 90th percentile absolute error: 3.4%.

Where does the model err? diagnostics/03_error_by_segment.png



Shows	Grey bars: MAPE per segment. Red line: mean signed % error (bias; + = over-prediction).
Finding	Accuracy (MAPE) is similar in every equipment, distance, weight and weekday segment (1.5–1.9%). Bias shows mild structure: short hauls (<250 mi) are under-predicted by 0.5% and the longest hauls over-predicted by 0.8%, a hint that the single distance exponent is slightly too steep at the extremes. The largest structure is by month (-1.0% to +2.0%): the remaining error is mostly about the market level of a period.
Decision	Acceptable (all segment biases within ±1%). A possible next step is a distance spline in stage 1. The month-level bias is the main risk for Nov–Dec, hence the trend checks.

Error over time diagnostics/04_error_over_time.png

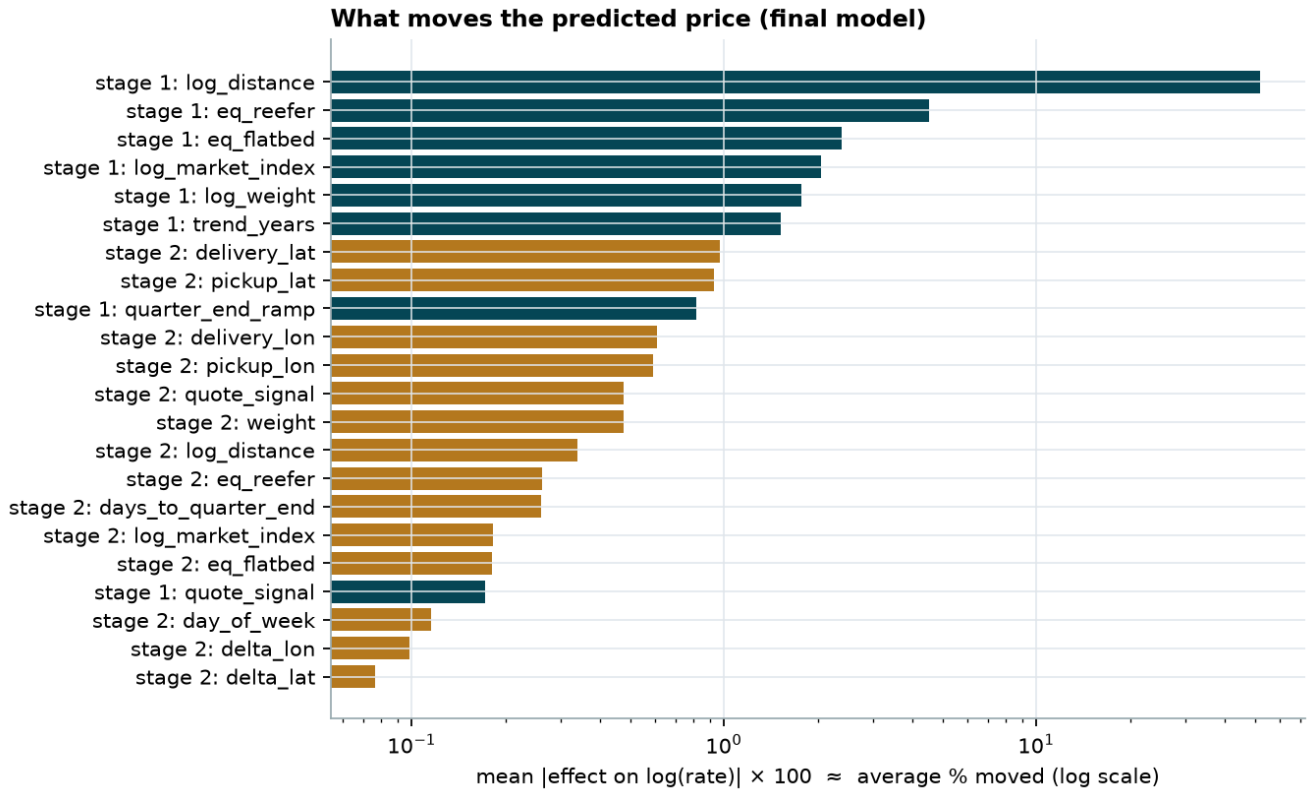


Shows Daily mean signed error and 7-day MAPE across the three evaluation windows (dashed lines = fold boundaries).

Finding Errors grow with distance from the training cut-off. Mean MAPE goes from 1.29% in the first two weeks of a fold to 1.83% in the last two, and the bias drifts by +0.6 / +1.6 / -1.1 points (folds 1/2/3) over the 8 weeks, in a direction that depends on how the market moved. Fold 2 (Jul–Aug) runs high: it was trained on only six months, when the trend was over-estimated (see checks).

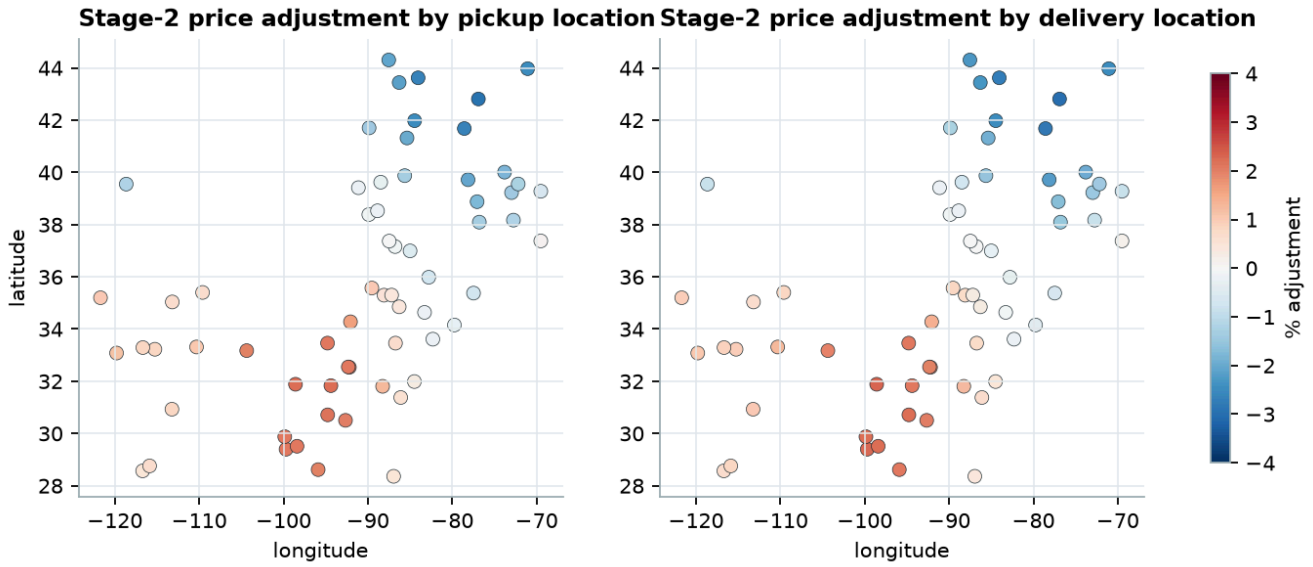
Decision Expect late-December predictions to be less accurate than early-November ones (roughly 1.8% vs 1.3% MAPE if the pattern repeats). Retraining weekly or monthly in production would keep the horizon short.

Feature importance diagnostics/05_feature_importance.png



Shows	Mean absolute contribution of each feature to log(rate) on the training rows: Ridge coefficient × centred feature for stage 1 (teal); exact SHAP values of the LightGBM member of the stage-2 ensemble (gold; the XGBoost and Extra Trees members learn the same residual and are not shown). Log scale.
Finding	Distance dominates (moves price by ~52% on average). Then equipment, weight, market index, quarter-end and trend in stage 1. Stage-2 corrections are small (mostly geography: pickup/delivery coordinates), refining the price by a few percent.
Decision	Confirms the design: the extrapolating drivers are in stage 1, the tree stage only refines.

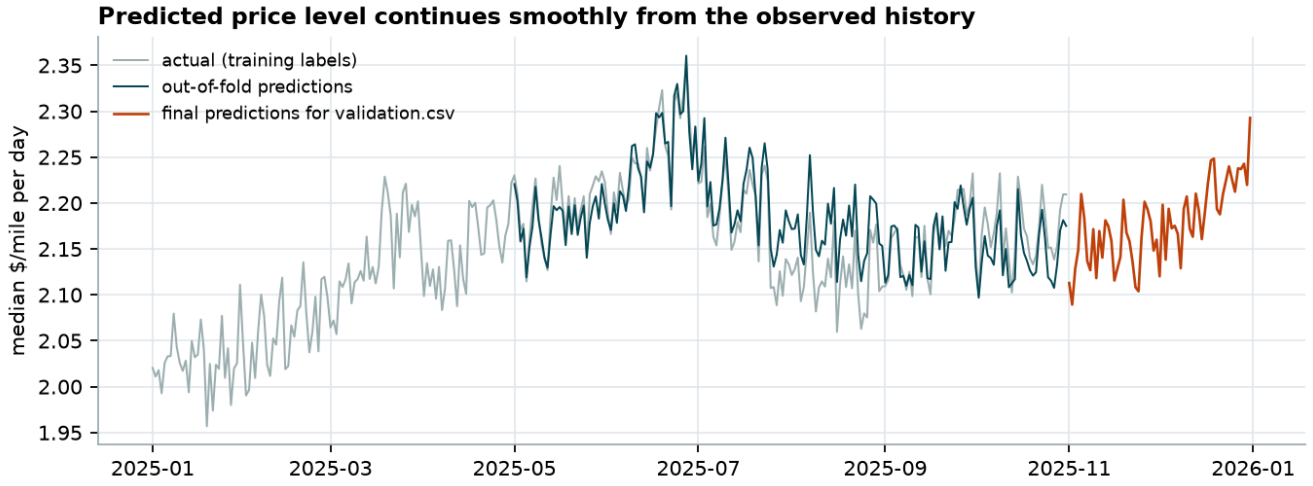
Learned geographic effects diagnostics/06_geography.png



Shows Average SHAP contribution of the pickup (left) and delivery (right) coordinates, per city, as a % price adjustment.

Finding Smooth regional patterns rather than city-by-city noise, which is what lets the model price cities it has never seen from their location.

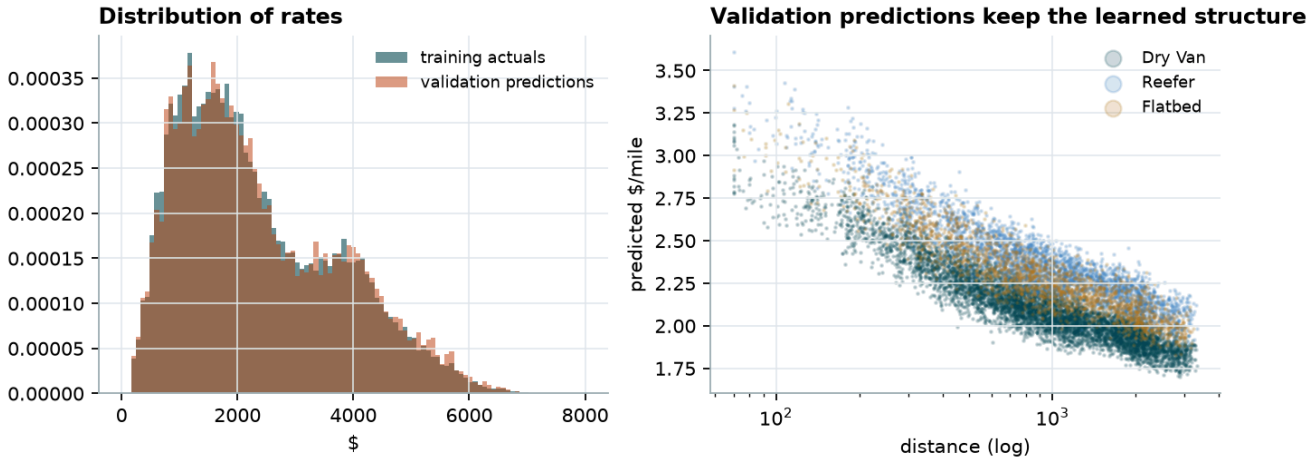
Do the Nov–Dec predictions join the history sensibly? diagnostics/07_continuity.png



Shows Median rate per mile per day: actual training labels, out-of-fold predictions, and the submitted predictions for November–December.

Finding The submitted predictions continue the October level and keep the same weekly rhythm; no jump at 1 November. Validation median $/mile 2.18 vs October actual 2.17.

Sanity of the submitted predictions diagnostics/08_validation_predictions.png

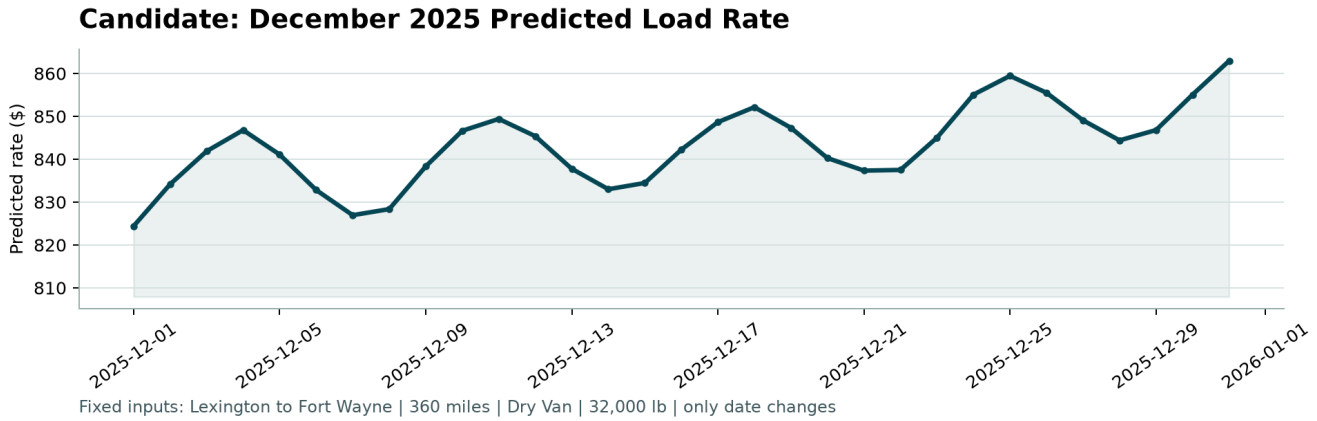


Shows Left: submitted validation predictions vs training actuals. Right: predicted $/mile by distance and equipment for the 12,000 validation loads.

Finding All 12,000 predictions are positive and finite ($194–$7,001); the distribution matches training, and the distance/equipment structure is preserved for every load including the unseen cities.

5. Required deliverable: December chart

Output of the provided score.py scorer_results/candidate_december.png



Shows Lexington → Fort Wayne, 360 miles, Dry Van, 32,000 lb, priced for every day of December 2025 by the submitted model.